

GestISP

Documentación técnica oficial

Sistema de gestión para proveedores de Internet

Guía de arquitectura, módulos, clases, métodos y operación

Generada el 25 de julio de 2026

Índice

Índice.....	2
1. Introducción.....	9
1.1 Cómo leer esta documentación.....	9
1.2 El sistema en cifras.....	9
2. Arquitectura y convenciones.....	10
2.1 Tecnologías.....	10
2.2 Organización del código.....	10
2.3 Convenciones del proyecto.....	10
Las rutas del panel llevan prefijo.....	10
Permisos: declarar y sincronizar.....	11
Listados con DataTables.....	11
Modales fuera de las tablas.....	11
Documentación en el propio código.....	11
Mensajes al usuario.....	11
Pruebas antes de dar por terminado.....	11
3. Módulos funcionales.....	12
3.1 Acceso, sesiones y perfil.....	12
3.2 Clientes y contratos.....	12
3.3 Facturación.....	12
3.4 Pagos y cajas.....	13
3.5 Órdenes técnicas.....	13
3.6 Inventario y almacenes.....	13
3.7 Red GPON: OLT y ONT.....	14
3.8 Red PPPoE: routers MikroTik.....	14
3.9 Notificaciones.....	14
3.10 Informes gerenciales.....	14
3.11 Usuarios, roles y permisos.....	15
3.12 Trazabilidad y auditoría.....	15
3.13 Infraestructura y utilidades.....	15
4. Referencia de clases y métodos.....	16
4.1 Acceso, sesiones y perfil.....	16
ConfirmPasswordController.....	16
ForgotPasswordController.....	16

LoginController	16
ProfileController.....	17
RegisterController	17
ResetPasswordController.....	17
VerificationController	17
SessionTracker	18
FailedLogin	18
UserSession	18
SweepUserSessions.....	19
ApplyUserTheme.....	19
CheckPermission	19
EnsureBranchSession	19
EnsureSuperadmin.....	19
TrackUserActivity	20
UserAgentParser	20
RecordFailedLogin.....	20
4.2 Clientes y contratos	20
ClientController.....	20
ContractCommentController	21
ContractController	21
ContractLinker.....	21
SnmpClient.....	22
ContractStatus	22
Client	22
Contract.....	23
ContractComment	24
ClientWelcome.....	24
TechnicalOrderCreatedClient.....	25
TechnicalOrderFinishedClient.....	26
ContractStatusMap	27
ColombiaLocations.....	27
ClientsExport.....	27
ContractsExport	27
NotifyClientInvoiceIssued	28
4.3 Facturación.....	28

AdditionalChargeController	28
InvoiceController.....	28
InvoiceGenerator	29
InvoiceNumerator	29
InvoiceVoider	30
MonthlyBillingRun.....	30
OverdueProcessor.....	30
PaymentRegistrar.....	30
InvoiceStatus.....	30
InvoiceType	31
PaymentStatus	31
ProrationMode.....	31
AditonalCharge.....	31
BillingRun	31
BranchBillingSetting.....	32
Invoice	32
InvoiceItem	33
InvoiceNumberingSequence	33
NotifyInvoiceReminders.....	33
GeneratePendingInvoicesPdf.....	33
InvoiceDueSoon	35
InvoiceGenerated.....	35
InvoiceOverdue.....	36
4.4 Pagos y cajas	37
CashRegisterController	37
CashRegisterTransactionController	38
PaymentController.....	38
CashRegister.....	39
CashRegisterTransaction.....	40
Payment	41
PaymentAudit	42
PaymentsExport.....	42
4.5 Órdenes técnicas.....	42
TechnicalOrderController	42
TechnicalOrder.....	43

TechnicalOrderMaterial	43
TechnicalOrderVerification	44
TechnicalOrderAssignedTechnician	44
OrderDetailMap	45
TechnicalOrdersReport	45
TechnicalOrdersExport.....	46
4.6 Inventario y almacenes	46
CategoryController.....	46
MaterialController	47
MaterialMovementController	47
WarehouseController	48
Category.....	48
Inventory.....	48
Material.....	48
MaterialMovement.....	49
Warehouse.....	49
MaterialsMovementsExport	49
4.7 Red GPON: OLT y ONT.....	50
OltController	50
OntController	50
OntImportController	51
OltOntDiscovery	52
OltSnmpService.....	52
OltSshService.....	53
OntPoller	53
LineProfile	53
Olt.....	53
Ont	54
OntImportRun.....	54
OntMetric.....	55
SrvProfile.....	55
VlanOlt	55
ImportOnt.....	55
OltSnmpProbe.....	55
PollOnt.....	55

SyncOltInterfaces	55
SyncOntPower.....	56
ImportOltOntS.....	56
4.8 Red PPPoE: routers MikroTik	57
PppoeAccountController	57
RouterController	58
MikrotikApiService	58
PppoeAccount.....	59
PppoeSessionMetric	59
Router	59
4.9 Notificaciones	60
NotificationController	60
LogGateway.....	60
MetaCloudGateway	60
RespetoCanales	60
WhatsAppChannel	60
WhatsAppGateway	61
WhatsAppMessage	61
PhoneNumber.....	61
4.10 Informes gerenciales.....	61
ManagementReportController	61
BillingReport.....	62
Granularity	62
GrowthReport	63
ProvisioningReport.....	63
ReportPeriod.....	63
4.11 Usuarios, roles y permisos	64
AdminController.....	64
BranchController.....	64
RoleController.....	65
UserController.....	65
Branch	66
User	66
SyncPermissions.....	69
PermissionLabels.....	69

RoleBasedMenuFilter.....	70
4.12 Trazabilidad y auditoría.....	70
AuditController	70
AuditLabels.....	70
AuditLogger.....	70
Audit.....	71
PruneAudits.....	71
AuditRequests.....	71
AuditServiceProvider	72
4.13 Infraestructura y utilidades.....	72
Controller	72
HomeController	72
PlanController	72
ServiceController.....	73
PppoePoller.....	73
PdfReport	73
Plan.....	74
Service.....	74
PollPppoe	74
TestWhatsAppTemplate	74
Authenticate	74
EncryptCookies.....	74
PreventRequestsDuringMaintenance.....	74
RedirectIfAuthenticated.....	75
TrimStrings	75
TrustHosts	75
TrustProxies	75
ValidateSignature.....	75
VerifyCsrfToken.....	75
AppServiceProvider.....	75
AuthServiceProvider	75
BroadcastServiceProvider	75
EventServiceProvider	76
RouteServiceProvider	76
PdfBranding.....	76

UnreadOrdersBadgeFilter	76
TransactionsReport	76
PersistDarkModePreference	77
5. Comandos de consola	78
5.1 El planificador	78
5.2 Comandos de Laravel de uso frecuente.....	78
6. Despliegue y operación.....	80
6.1 Procedimiento de despliegue	80
6.2 Errores frecuentes al desplegar	80
6.3 Requisitos del servidor.....	80
7. Pruebas automatizadas.....	81

1. Introducción

GestISP es el sistema con el que un proveedor de Internet administra su operación completa: clientes y contratos, facturación y cobros, trabajo de campo de los técnicos, inventario de materiales y equipos, y la red de fibra (OLT y ONT) y PPPoE.

Este documento es la referencia oficial del proyecto. Está pensado para que una persona que se incorpore al equipo entienda cómo está construido el sistema, dónde está cada cosa, por qué se tomaron ciertas decisiones y qué comandos debe usar en cada situación.

Todo el contenido se generó a partir del código fuente en su estado actual: las clases, los métodos y sus descripciones se extrajeron por reflexión, no se escribieron de memoria.

1.1 Cómo leer esta documentación

- El capítulo 2 describe la arquitectura y las convenciones. Es la lectura obligatoria antes de tocar código.
- El capítulo 3 explica módulo por módulo qué hace el sistema y las reglas de negocio que hay detrás.
- El capítulo 4 es la referencia completa de clases y métodos, agrupada por módulo y por capa.
- Los capítulos 5 y 6 reúnen los comandos de consola y el procedimiento de despliegue.

1.2 El sistema en cifras

Elemento	Cantidad
Rutas registradas	255
Migraciones	72
Vistas Blade	116
Clases documentadas	168
Métodos públicos	1.055
Archivos de prueba	38
Pruebas automatizadas	302 (1.024 comprobaciones)

2. Arquitectura y convenciones

2.1 Tecnologías

Componente	Tecnología
Lenguaje	PHP 8.3
Framework	Laravel 10.48
Base de datos	MySQL 8
Panel de administración	AdminLTE 3.14 (jeroennoten/laravel-adminlte)
Interfaz	Bootstrap 4 en el panel · Bootstrap 5 en las pantallas de acceso
Tablas	DataTables 1.13.6 (lado cliente, tema Bootstrap 4)
Permisos	spatie/laravel-permission 6
PDF	barryvdh/laravel-dompdf 3
Excel	maatwebsite/excel 3
SSH a equipos de red	phpseclib/phpseclib 3
API MikroTik	evilfreelancer/routeros-api-php 1.7
SNMP	Extensión nativa de PHP (php-snmp)
Códigos de barras	milon/barcode 11

2.2 Organización del código

El proyecto sigue la estructura estándar de Laravel con algunas carpetas propias:

Carpeta	Qué contiene
app/Http/Controllers	Reciben la petición HTTP y devuelven la respuesta. No contienen lógica de negocio.
app/Services	Lógica de negocio y comunicación con equipos de red (SSH, SNMP, API MikroTik).
app/Billing	Todo lo relativo a facturación: servicios, enumeraciones de estado y eventos.
app/Models	Modelos Eloquent: representan las tablas y sus relaciones.
app/Console/Commands	Comandos de consola (sondeos, importaciones, mantenimiento).
app/Jobs	Trabajos que se ejecutan en segundo plano a través de la cola.
app/Http/Middleware	Filtros que se aplican a las peticiones (sesión, permisos, auditoría).
app/Notifications	Avisos por correo, WhatsApp y dentro del sistema.
app/Reports	Cálculos de los informes gerenciales.
app/Support	Utilidades transversales (PDF, teléfonos, ubicaciones).
resources/views/gestisp	Vistas Blade del panel, organizadas por módulo.
resources/lang/es	Textos de error y validación en español.
config	Configuración. Propias del proyecto: olt_snmp.php, audit.php, notifications.php, adminlte.php.
tests/Feature	Pruebas automatizadas, agrupadas por módulo.

2.3 Convenciones del proyecto

Estas reglas no son preferencias de estilo: cada una nació de un problema real. Respetarlas evita repetirlo.

Las rutas del panel llevan prefijo

La mayoría de rutas del panel están bajo el prefijo `gestisp/`. Por eso las URL NUNCA se escriben a mano en JavaScript: se generan con `route()` en Blade y se pasan al script. Escribirlas a mano produce errores 404 en producción que no aparecen en desarrollo.

Permisos: declarar y sincronizar

Cada acción protegida declara su permiso en el constructor del controlador con `check.permission:modulo.accion`. Al agregar uno nuevo hay que ejecutar `php artisan permissions:sync` en el despliegue, o la pantalla quedará inaccesible. Para endpoints compartidos entre módulos se usa solo `auth`, porque el middleware no admite permisos alternativos.

Listados con DataTables

Los listados usan DataTables del lado del cliente (CDN 1.13.6, tema Bootstrap 4, traducción es-ES). El controlador entrega la colección completa con `->get()` y las relaciones precargadas con `with()` para evitar consultas N+1. Excepción: la auditoría, que pagina en el servidor porque crece sin límite.

Modales fuera de las tablas

Los modales de detalle van FUERA del elemento `<table>`. DataTables reordena y pagina las filas, y un modal declarado dentro de una celda queda inaccesible.

Documentación en el propio código

El proyecto se documenta en español dentro del código: bloque PHPDoc en cada clase y método, y comentarios de sección en las vistas Blade. Los comentarios explican POR QUÉ se hizo así, no solo qué hace.

Mensajes al usuario

Los textos de error y validación viven en `resources/lang/es` (`auth.php`, `passwords.php`, `validation.php`). No se escriben mensajes de autenticación sueltos en el código.

Pruebas antes de dar por terminado

Toda funcionalidad nueva se acompaña de pruebas en `tests/Feature`. Se ejecutan con `php artisan test` sobre la base `gestisp_testing`. Actualmente hay 302 pruebas y deben quedar todas en verde.

3. Módulos funcionales

Cada módulo se describe con su propósito y las reglas de negocio que conviene conocer antes de modificarlo. La referencia detallada de sus clases está en el capítulo 4.

3.1 Acceso, sesiones y perfil

Controla quién entra al sistema, con qué rol y en qué sucursal, y mantiene la trazabilidad de cada sesión.

Puntos clave:

- Una sesión válida necesita TRES cosas: usuario autenticado, `branch_id` y `current_role_id`. Los dos últimos se eligen en el formulario de acceso (el usuario selecciona la sucursal) y determinan qué puede hacer.
- Si la sesión pierde ese contexto (restablecer contraseña, cookie de "recordarme"), el middleware `EnsureBranchSession` lo repone desde la sucursal que el usuario tenía seleccionada. Sin él, el usuario quedaba encerrado con un error 403 en el propio panel.
- La sesión se cierra sola tras 15 minutos de inactividad (configurable con `SESSION_INACTIVITY_TIMEOUT`). Los sondeos automáticos del navegador NO cuentan como actividad: si contaran, una pestaña olvidada mantendría la sesión abierta indefinidamente.
- Cada usuario administra sus propios datos en `/perfil`: foto, datos personales y contraseña. Cambiar la contraseña cierra las sesiones abiertas en otros equipos.

3.2 Clientes y contratos

Registro de los clientes del ISP y de los contratos de servicio que se les asocian. Es el punto de partida de la facturación y de las órdenes técnicas.

Puntos clave:

- Al crear un contrato el sistema genera automáticamente una orden técnica de instalación ("Instalación de servicio (creación automática)") y envía la bienvenida al cliente por correo y WhatsApp.
- El detalle del contrato reúne cuatro pestañas: estado de cuenta (facturas), cargos adicionales, historial de operaciones (órdenes técnicas, con su comprobante en PDF) y comentarios internos.
- La columna `contracts.status` recibe valores desde dos vocabularios distintos (facturación y formulario manual). Para interpretarla siempre debe usarse `ContractStatusMap`, nunca comparar cadenas a mano.

3.3 Facturación

Emisión mensual de facturas, cargos adicionales, anulaciones y numeración fiscal por sucursal.

Puntos clave:

- La lógica de negocio vive en `app/Billing/Services` (`MonthlyBillingRun`, `InvoiceGenerator`, `OverdueProcessor`, `PaymentRegistrar`). Los controladores solo manejan HTTP: no debe volver a meterse lógica de negocio en ellos.
- Los estados son enumeraciones canónicas en `app/Billing/Enums` (`InvoiceStatus`, `ContractStatus`, `PaymentStatus`). Nunca escribir los estados como texto suelto.

- Las facturas NO se eliminan: se anulan (InvoiceVoider) y queda registro. La numeración se asigna con InvoiceNumerator usando bloqueo de fila, de modo que dos procesos simultáneos no repitan número.
- Las reglas de cobro (prorrateso, días de gracia, facturas vencidas permitidas) se configuran por sucursal en branch_billing_settings.
- El dinero se guarda en DECIMAL(15,2). La deuda de un contrato se obtiene con Contract::outstandingBalance(), que suma los saldos de las facturas abiertas.

3.4 Pagos y cajas

Registro de los cobros a clientes y control de las cajas por punto de atención, con cuadro por período.

Puntos clave:

- Todo cobro exige tener una caja abierta, sin importar el medio de pago. Lo garantiza PaymentRegistrar.
- Al registrar un pago de un contrato suspendido, el sistema genera automáticamente una orden técnica de reconexión.
- El resumen y cuadro de cajas está en cash_register.summary, protegido por su propio permiso.

3.5 Órdenes técnicas

Trabajos de campo: instalaciones, traslados, reconexiones, cortes e incidencias. Es el flujo que conecta la oficina con los técnicos.

Puntos clave:

- Ciclo de vida: Pendiente → Asignada → Prefinalizada → Cerrada. El técnico puede rechazarla y un supervisor puede devolverla a Pendiente.
- Las órdenes de INSTALACIÓN exigen registrar material: no se pueden procesar sin al menos un material o equipo. Se valida en el servidor, no solo en el navegador.
- El técnico descuenta el material de SU almacén personal (cada técnico tiene un Warehouse con user_id). Los equipos se descuentan por número de serie; los consumibles por cantidad.
- Al procesar la orden se exige la firma del cliente, capturada en pantalla con el dedo o un lápiz táctil, y se guarda como imagen.
- Cada orden cerrada tiene un comprobante en PDF (materiales, solución, comentarios, evidencia fotográfica y firma) que se puede ver dentro de la misma página o descargar.

3.6 Inventario y almacenes

Control de materiales y equipos: existencias por almacén, movimientos (entradas, salidas y traslados) e historial.

Puntos clave:

- Los EQUIPOS (is_equipment) se rastrean por número de serie: una fila de inventario por serial. Los CONSUMIBLES se rastrean por cantidad.
- Cada movimiento actualiza el stock dentro de una transacción: si algo falla, se revierte todo.
- Cada técnico tiene su propio almacén, que es de donde sale el material que reporta en las órdenes.

3.7 Red GPON: OLT y ONT

Gestión de las OLT Huawei y de las ONT de los clientes: alta, monitoreo, diagnóstico y operaciones remotas.

Puntos clave:

- LECTURA por SNMP, ESCRITURA por SSH. La consulta de una ONT por SNMP tarda milisegundos frente a segundos por consola; la consola queda solo para lo que SNMP no puede hacer (activar, mover, reiniciar, CATV).
- El diálogo por consola es ADAPTATIVO: reacciona a lo que responde el equipo (prompt, confirmaciones, paginación) en lugar de suponer por modelo. Antes se adivinaba según el modelo y la MA5608T se desincronizaba.
- Los OIDs y sus escalas están en `config/olt_snmp.php` y fueron verificados contra equipos reales. Se comprueban con `php artisan olt:snmp-probe`.
- La ubicación de una ONT (slot/puerto) se deduce del `ifIndex` cuando la OLT no publica las descripciones de interfaz, como ocurre en la MA5608T.
- El historial de conexión (última conexión, última caída y su causa) se lee de la tabla de control de la ONT. Las causas verificadas son: `dying-gasp` (corte de energía), `LOSi/LOBi`, `LOFi` y `Operator check failure`.

3.8 Red PPPoE: routers MikroTik

Cuentas PPPoE de los clientes sobre routers MikroTik: alta, estado en tiempo real y consumo.

Puntos clave:

- Se usa la API REST de RouterOS. Cada sesión activa crea una interfaz dinámica cuyos contadores permiten graficar el consumo.
- El tráfico se muestrea cada 5 minutos y se guarda en `pppoe_session_metrics`.
- Atención: lo que el router llama "rx" es la SUBIDA del cliente. Al reconectarse una sesión los contadores se reinician, por lo que se descartan las diferencias negativas.

3.9 Notificaciones

Avisos automáticos a clientes y técnicos por correo, WhatsApp y dentro del propio sistema.

Puntos clave:

- Los envíos van en cola: una notificación lenta o un proveedor caído nunca deben bloquear la operación que la disparó.
- El proveedor de WhatsApp es intercambiable. El oficial es la API de Meta; existe un modo simulado para pruebas.
- Las plantillas de WhatsApp deben estar aprobadas en Meta y su nombre e idioma deben coincidir exactamente con lo declarado en el código.

3.10 Informes gerenciales

Tableros de dirección: crecimiento, órdenes técnicas, facturación y aprovisionamiento.

Puntos clave:

- Algunos indicadores (altas, bajas, tiempo de resolución) son APROXIMACIONES calculadas a partir de las columnas disponibles, porque el esquema no guarda esas fechas de forma explícita. Está documentado en el propio informe.

3.11 Usuarios, roles y permisos

Alta de usuarios del sistema, asignación de roles por sucursal y control de lo que cada rol puede hacer.

Puntos clave:

- El permiso se evalúa contra el ROL ACTIVO de la sesión (el de la sucursal en la que se entró), no contra todos los roles del usuario. El mismo criterio aplica el middleware y el filtro del menú.
- Los permisos se declaran en los controladores con `check.permission:modulo.accion` y se crean en la base con `php artisan permissions:sync`. Este comando debe ejecutarse en CADA despliegue que agregue permisos.
- Un usuario puede inhabilitarse sin borrarlo; queda expulsado de sus sesiones abiertas de inmediato.

3.12 Trazabilidad y auditoría

Registro de todo lo que hacen los usuarios en el sistema, para auditorías. Pantalla reservada al superadministrador.

Puntos clave:

- Tres fuentes: cambios en cualquier modelo (escucha global de Eloquent), acciones que no cambian datos (middleware sobre las peticiones) y accesos (eventos de login, logout e intentos fallidos).
- Se audita lo que hace una PERSONA y se descarta el ruido de la MÁQUINA. El sondeo de ONTs reescribe la potencia de miles de equipos cada 5 minutos: sin ese filtro se generarían cientos de miles de registros diarios que esconderían la actividad real.
- Las contraseñas, tokens y comunidades SNMP nunca se almacenan: se sustituyen por asteriscos.
- El acceso exige el ROL superadministrador, no un permiso, para que no pueda concederse por error desde el módulo de roles.
- La pantalla usa paginación de servidor (no DataTables de cliente) porque la bitácora crece sin límite.

3.13 Infraestructura y utilidades

Piezas transversales: proveedores de servicios, utilidades de PDF, exportaciones a Excel y trabajos en segundo plano.

Puntos clave:

- Todos los PDF (excepto las facturas, que conservan su formato fiscal) se construyen con `App\Support\PdfBranding::make()`, que aplica el encabezado de la sucursal, el tamaño carta y la paginación.
- Las exportaciones a Excel están en `app/Exports` y respetan los filtros de la pantalla desde la que se lanzan.

4. Referencia de clases y métodos

Listado completo de las clases del sistema con sus métodos públicos, agrupadas por módulo funcional y, dentro de cada uno, por capa. La descripción de cada elemento procede del comentario del propio código.

4.1 Acceso, sesiones y perfil

Controladores

ConfirmPasswordController

App\Http\Controllers\Auth\ConfirmPasswordController

- **__construct()**
Create a new controller instance.
- **showConfirmForm()**
Display the password confirmation view.
- **confirm(\$request)**
Confirm the given user's password.
- **redirectPath()**
Get the post register / login redirect path.

ForgotPasswordController

App\Http\Controllers\Auth\ForgotPasswordController

- **showLinkRequestForm()**
Display the form to request a password reset link.
- **sendResetLinkEmail(\$request)**
Send a reset link to the given user.
- **broker()**
Get the broker to be used during password reset.

LoginController

App\Http\Controllers\Auth>LoginController

- **__construct(\$tracker)**
- **getBranches(\$request)**
- **logout(\$request)**
- **showLoginForm()**
Show the application's login form.
- **login(\$request)**
Handle a login request to the application.
- **username()**
Get the login username to be used by the controller.
- **redirectPath()**
Get the post register / login redirect path.

- **maxAttempts()**
Get the maximum number of attempts to allow.
- **decayMinutes()**
Get the number of minutes to throttle for.

ProfileController

App\Http\Controllers\ProfileController

Perfil del usuario autenticado.

- **__construct(\$tracker)**
- **edit()**
Pantalla del perfil: datos, seguridad y sesiones recientes.
- **update(\$request)**
Actualiza los datos personales.
- **updatePassword(\$request)**
Cambia la contraseña.
- **updatePhoto(\$request)**
Sube o reemplaza la foto de perfil.
- **destroyPhoto()**
Quita la foto y vuelve al avatar con las iniciales.

RegisterController

App\Http\Controllers\Auth\RegisterController

- **__construct()**
Create a new controller instance.
- **showRegistrationForm()**
Show the application registration form.
- **register(\$request)**
Handle a registration request for the application.
- **redirectPath()**
Get the post register / login redirect path.

ResetPasswordController

App\Http\Controllers\Auth\ResetPasswordController

- **showResetForm(\$request)**
Display the password reset view for the given token.
- **reset(\$request)**
Reset the given user's password.
- **broker()**
Get the broker to be used during password reset.
- **redirectPath()**
Get the post register / login redirect path.

VerificationController

App\Http\Controllers\Auth\VerificationController

- **__construct()**
Create a new controller instance.
- **show(\$request)**
Show the email verification notice.
- **verify(\$request)**
Mark the authenticated user's email address as verified.
- **resend(\$request)**
Resend the email verification notification.
- **redirectPath()**
Get the post register / login redirect path.

Servicios

SessionTracker

App\Services\SessionTracker

Registro de la trazabilidad de sesiones.

- **start(\$user, \$request, \$branchId)**
Registra un inicio de sesión.
- **touch(\$request)**
Actualiza la última actividad de la sesión en curso.
- **end(\$traceId, \$reason)**
Marca el cierre de la sesión en curso.
- **traceIdActual(\$request)**
Id de la fila de trazabilidad guardado en la sesión actual.
- **forceClose(\$session)**
Cierra de forma remota una sesión concreta.
- **forceCloseAllFor(\$user, \$exceptTraceId)**
Cierra de forma remota todas las sesiones activas de un usuario.
- **recordFailure(\$email, \$request)**
Registra un intento de inicio de sesión fallido.
- **sesionActual(\$request)**
La fila de sesión correspondiente a la sesión viva actual.

Modelos

FailedLogin

App\Models\FailedLogin

Intento de inicio de sesión fallido.

- **user()**

UserSession

App\Models\UserSession

Registro de una sesión de un usuario.

- **user()**
- **branch()**
- **staleThreshold()**
Momento a partir del cual una sesión sin actividad se considera caducada.
- **scopeActive(\$query)**
Sesiones vivas: sin salida y con actividad reciente.
- **scopeEnded(\$query)**
Sesiones terminadas: con salida registrada, o sin actividad dentro del tiempo de vida (caducadas aunque el barrido aún no las haya marcado).
- **isActive()**
¿La sesión sigue viva ahora mismo?
- **estadoLegible()**
Motivo de cierre legible, resolviendo también la caducidad que el barrido todavía no haya escrito.
- **duracionMinutos()**
Duración de la sesión en minutos (hasta la salida o, si sigue viva, hasta ahora).

Comandos de consola

SweepUserSessions

App\Console\Commands\SweepUserSessions

Cierra formalmente las sesiones que caducaron por inactividad.

- **handle()**

Middleware

ApplyUserTheme

App\Http\Middleware\ApplyUserTheme

Aplica a la sesión el tema (claro/oscuro) guardado en el usuario.

- **handle(\$request, \$next)**

CheckPermission

App\Http\Middleware\CheckPermission

Verifica que el ROL ACTIVO de la sesión (el elegido al entrar a una sucursal) tenga el permiso exigido por la ruta.

- **handle(\$request, \$next, \$permission)**

EnsureBranchSession

App\Http\Middleware\EnsureBranchSession

Restaura la sucursal y el rol activos de la sesión cuando faltan.

- **handle(\$request, \$next)**

EnsureSuperadmin

App\Http\Middleware\EnsureSuperadmin

Reserva una pantalla al superadministrador.

- **handle(\$request, \$next)**

TrackUserActivity

App\Http\Middleware\TrackUserActivity

Mantiene viva la trazabilidad de la sesión en curso y aplica los cierres remotos.

- **__construct(\$tracker)**
- **handle(\$request, \$next)**

Utilidades

UserAgentParser

App\Support\UserAgentParser

Interpreta la cadena User-Agent del navegador.

- **parse(\$userAgent)**
- **resumen(\$userAgent)**
Etiqueta corta para mostrar: "Chrome · Windows 10/11".

Escuchadores de eventos

RecordFailedLogin

App\Listeners\RecordFailedLogin

Anota en la trazabilidad cada intento de inicio de sesión fallido.

- **__construct(\$tracker)**
- **handle(\$event)**

4.2 Clientes y contratos

Controladores

ClientController

App\Http\Controllers\ClientController

- **__construct()**
- **searchView(\$request)**
Buscador de clientes.
- **search(\$request)**
Compatibilidad: el buscador antiguo enviaba por POST. Se redirige al buscador nuevo (GET) conservando el término.
- **index(\$request)**
Display a listing of the resource.
- **export()**
- **create()**
Show the form for creating a new resource.
- **store(\$request)**
Store a newly created resource in storage.

- **show(\$client)**
Display the specified resource.
- **edit(\$client)**
Show the form for editing the specified resource.
- **update(\$request, \$client)**
Update the specified resource in storage.
- **destroy(\$client)**
Remove the specified resource from storage.

ContractCommentController

App\Http\Controllers\ContractCommentController

Comentarios/notas internas de un contrato.

- **__construct()**
- **store(\$request, \$contract)**
Guarda un comentario nuevo en el contrato.
- **destroy(\$comment)**
Elimina un comentario del contrato.

ContractController

App\Http\Controllers\ContractController

- **__construct()**
- **index(\$request)**
Display a listing of the resource.
- **create(\$client)**
Show the form for creating a new resource.
- **store(\$request)**
Store a newly created resource in storage.
- **show(\$contract)**
Display the specified resource.
- **edit(\$contract)**
Show the form for editing the specified resource.
- **update(\$request, \$contract)**
Update the specified resource in storage.
- **destroy(\$contract)**
Remove the specified resource from storage.
- **export()**

Servicios

ContractLinker

App\Services\ContractLinker

Vinculación de equipos con contratos.

- **linkOnt(\$ont, \$contractId)**

Asocia una ONT a un contrato.

- **unlinkOnt(\$ont)**
Quita la asociación de una ONT con su contrato.
- **linkPppoe(\$cuenta, \$contractId)**
Asocia una cuenta PPPoE a un contrato.
- **unlinkPppoe(\$cuenta)**

SnmpClient

App\Services\Snmp\SnmpClient

Cliente SNMP de bajo nivel para las OLTs.

- **__construct(\$host, \$community, \$timeout, \$retries, \$maxRepetitions)**
- **isAvailable()**
¿Está disponible la extensión SNMP de PHP?
- **forOlt(\$olt)**
Construye el cliente a partir de los datos de la OLT. Devuelve null si falta la extensión SNMP o la OLT no tiene community de lectura.
- **getMany(\$oids)**
Consulta varios OIDs en UNA sola petición.
- **get(\$oid)**
Consulta un OID puntual. Devuelve null si no responde.
- **walk(\$oid)**
Recorre una tabla completa con GETBULK.
- **isReachable()**
Comprueba que la OLT responde por SNMP (sysUpTime).
- **close()**

Enumeraciones de facturación

ContractStatus

App\Billing\Enums\ContractStatus

Estados del ciclo de vida de un contrato.

- **billable()**
Estados que se incluyen en la generación de facturas mensuales (con servicio activo o en riesgo, aún no cortado).
- **cases()**
- **from(\$value)**
- **tryFrom(\$value)**

Modelos

Client

App\Models\Client

- **branch()**

- **user()**
- **contracts()**
- **fullName()**
Nombre completo del cliente, para los saludos.
- **routeNotificationForMail()**
Dirección de correo a la que se le notifica.
- **routeNotificationForWhatsApp()**
Número al que se le envía WhatsApp (el principal; el adicional queda de respaldo por si el principal está vacío).
- **factory(\$count, \$state)**
Get a new factory instance for the model.
- **notifications()**
Get the entity's notifications.
- **readNotifications()**
Get the entity's read notifications.
- **unreadNotifications()**
Get the entity's unread notifications.
- **notify(\$instance)**
Send the given notification.
- **notifyNow(\$instance, \$channels)**
Send the given notification immediately.
- **routeNotificationFor(\$driver, \$notification)**
Get the notification routing information for the given driver.

Contract

App\Models\Contract

- **client()**
Relación con la tabla Clients (Clientes)
- **plan()**
Relación con la tabla Plans
- **user()**
Relación con la tabla Users
- **additionalCharges()**
- **comments()**
Comentarios/notas internas sobre el contrato (más recientes primero).
- **branch()**
- **ont()**
- **pppoeAccounts()**
Cuentas PPPoE del contrato.
- **invoices()**
Relación con las facturas del contrato
- **overdueInvoices()**

Relación con facturas vencidas específicamente

- **pendingInvoices()**
Relación con facturas pendientes
- **openInvoices()**
Facturas abiertas (admiten pago): pendientes, parciales, con riesgo de corte y vencidas.
- **outstandingBalance()**
Saldo total adeudado del contrato: la suma de los saldos de sus facturas abiertas. Es el estado de cuenta que reemplaza al patrón de absorción de vencidas (fase 4): las facturas quedan abiertas e independientes y la deuda se lee aquí.
- **factory(\$count, \$state)**
Get a new factory instance for the model.

ContractComment

App\Models\ContractComment

Comentario/nota interna sobre un contrato.

- **contract()**
Contrato sobre el que se hizo el comentario */
- **user()**
Usuario que escribió el comentario */
- **factory(\$count, \$state)**
Get a new factory instance for the model.

Notificaciones

ClientWelcome

App\Notifications\ClientWelcome

Bienvenida al cliente cuando se le crea/asigna un contrato.

- **__construct(\$contract)**
- **via(\$notifiable)**
- **toMail(\$notifiable)**
- **toWhatsApp(\$notifiable)**
- **onConnection(\$connection)**
Set the desired connection for the job.
- **onQueue(\$queue)**
Set the desired queue for the job.
- **allOnConnection(\$connection)**
Set the desired connection for the chain.
- **allOnQueue(\$queue)**
Set the desired queue for the chain.
- **delay(\$delay)**
Set the desired delay in seconds for the job.
- **afterCommit()**
Indicate that the job should be dispatched after all database transactions have committed.

- **beforeCommit()**
Indicate that the job should not wait until database transactions have been committed before dispatching.
- **through(\$middleware)**
Specify the middleware the job should be dispatched through.
- **chain(\$chain)**
Set the jobs that should run if this job is successful.
- **prependToChain(\$job)**
Prepend a job to the current chain so that it is run after the currently running job.
- **appendToChain(\$job)**
Append a job to the end of the current chain.
- **dispatchNextJobInChain()**
Dispatch the next job on the chain.
- **invokeChainCatchCallbacks(\$e)**
Invoke all of the chain's failed job callbacks.

TechnicalOrderCreatedClient

App\Notifications\TechnicalOrderCreatedClient

Aviso al cliente de que se creó una orden técnica para su servicio.

- **__construct(\$order)**
- **via(\$notifiable)**
- **toMail(\$notifiable)**
- **toWhatsApp(\$notifiable)**
- **onConnection(\$connection)**
Set the desired connection for the job.
- **onQueue(\$queue)**
Set the desired queue for the job.
- **allOnConnection(\$connection)**
Set the desired connection for the chain.
- **allOnQueue(\$queue)**
Set the desired queue for the chain.
- **delay(\$delay)**
Set the desired delay in seconds for the job.
- **afterCommit()**
Indicate that the job should be dispatched after all database transactions have committed.
- **beforeCommit()**
Indicate that the job should not wait until database transactions have been committed before dispatching.
- **through(\$middleware)**
Specify the middleware the job should be dispatched through.
- **chain(\$chain)**
Set the jobs that should run if this job is successful.

- **prependToChain(\$job)**
Prepend a job to the current chain so that it is run after the currently running job.
- **appendToChain(\$job)**
Append a job to the end of the current chain.
- **dispatchNextJobInChain()**
Dispatch the next job on the chain.
- **invokeChainCatchCallbacks(\$e)**
Invoke all of the chain's failed job callbacks.

TechnicalOrderFinishedClient

App\Notifications\TechnicalOrderFinishedClient

Aviso al cliente de que su orden técnica quedó finalizada.

- **__construct(\$order)**
- **via(\$notifiable)**
- **toMail(\$notifiable)**
- **toWhatsApp(\$notifiable)**
- **onConnection(\$connection)**
Set the desired connection for the job.
- **onQueue(\$queue)**
Set the desired queue for the job.
- **allOnConnection(\$connection)**
Set the desired connection for the chain.
- **allOnQueue(\$queue)**
Set the desired queue for the chain.
- **delay(\$delay)**
Set the desired delay in seconds for the job.
- **afterCommit()**
Indicate that the job should be dispatched after all database transactions have committed.
- **beforeCommit()**
Indicate that the job should not wait until database transactions have been committed before dispatching.
- **through(\$middleware)**
Specify the middleware the job should be dispatched through.
- **chain(\$chain)**
Set the jobs that should run if this job is successful.
- **prependToChain(\$job)**
Prepend a job to the current chain so that it is run after the currently running job.
- **appendToChain(\$job)**
Append a job to the end of the current chain.
- **dispatchNextJobInChain()**
Dispatch the next job on the chain.
- **invokeChainCatchCallbacks(\$e)**

Invoke all of the chain's failed job callbacks.

Informes

ContractStatusMap

App\Reports\Support\ContractStatusMap

Unifica los estados de contrato para los informes.

- **vigentes()**
Estados que cuentan como cliente con servicio contratado.
- **bajas()**
Estados que representan una baja definitiva.
- **conServicio()**
Estados con servicio efectivamente prestándose.
- **sqlGrupo(\$columna)**
Expresión SQL que traduce contracts.status al grupo canónico.
- **etiqueta(\$grupo)**
- **color(\$grupo)**
- **grupos()**
- **estadosSinClasificar(\$branchId)**
Estados presentes en la base que no encajan en ningún grupo.

Utilidades

ColombiaLocations

App\Support\ColombiaLocations

Catálogo local de los 32 departamentos y 1.104 municipios de Colombia.

- **all()**
- **departmentsWithMunicipalities()**

Exportaciones

ClientsExport

App\Exports\ClientsExport

- **query()**
- **download(\$fileName, \$writerType, \$headers)**
- **store(\$filePath, \$disk, \$writerType, \$diskOptions)**
- **queue(\$filePath, \$disk, \$writerType, \$diskOptions)**
- **raw(\$writerType)**
- **toResponse(\$request)**
Create an HTTP response that represents the object.

ContractsExport

App\Exports\ContractsExport

- **query()**
- **headings()**
- **map(\$contract)**
- **download(\$fileName, \$writerType, \$headers)**
- **store(\$filePath, \$disk, \$writerType, \$diskOptions)**
- **queue(\$filePath, \$disk, \$writerType, \$diskOptions)**
- **raw(\$writerType)**
- **toResponse(\$request)**

Create an HTTP response that represents the object.

Escuchadores de eventos

NotifyClientInvoiceIssued

App\Listeners\NotifyClientInvoiceIssued

Avisa al cliente por correo y WhatsApp cuando se emite su factura.

- **handle(\$event)**

4.3 Facturación

Controladores

AdditionalChargeController

App\Http\Controllers\AdditionalChargeController

- **__construct()**
- **index()**
Display a listing of the resource.
- **create()**
Show the form for creating a new resource.
- **store(\$request)**
Store a newly created resource in storage.
- **show(\$additionalCharge)**
Display the specified resource.
- **edit(\$additionalCharge)**
Show the form for editing the specified resource.
- **update(\$request, \$additionalCharge)**
Update the specified resource in storage.
- **destroy(\$additionalCharge)**
Remove the specified resource from storage.

InvoiceController

App\Http\Controllers\InvoiceController

Controlador de Facturas

- **__construct()**

- **index(\$overdueProcessor)**
Display a listing of the resource.
- **create()**
Show the form for creating a new resource.
- **store(\$request)**
Store a newly created resource in storage.
- **show(\$invoice)**
Display the specified resource.
- **edit(\$invoice)**
Show the form for editing the specified resource.
- **update(\$request, \$invoice)**
Update the specified resource in storage.
- **destroy(\$invoice)**
Remove the specified resource from storage.
- **voidInvoice(\$request, \$invoice, \$voider)**
Anula una factura (nunca se elimina: cambia a estado Anulada con motivo, usuario y fecha). Las reglas de negocio viven en InvoiceVoider.
- **generateInvoices(\$billingRun)**
Ejecuta la corrida de facturación mensual de la sucursal activa. Toda la lógica vive en MonthlyBillingRun / InvoiceGenerator / OverdueProcessor (app/Billing/Services).
- **billingRuns()**
Reporte gerencial de corridas de facturación de la sucursal: cada generación con sus conteos y totales facturados.
- **downloadInvoicePdf(\$id)**
Método para crear PDF
- **generatePendingInvoicesPdf()**
PDF Masivo
- **checkPdfStatus(\$request)**

Servicios de facturación

InvoiceGenerator

App\Billing\Services\InvoiceGenerator

Generador de la factura mensual de UN contrato.

- **__construct(\$numerator)**
- **generateForContract(\$contract, \$today, \$userId)**
Genera la factura del período actual para un contrato.

InvoiceNumerator

App\Billing\Services\InvoiceNumerator

Asignación de numeración formal a las facturas.

- **assign(\$invoice)**
Asigna número a la factura y la persiste.

InvoiceVoider

App\Billing\Services\InvoiceVoider

Anulación formal de facturas.

- **__construct(\$overdueProcessor)**
- **void(\$invoice, \$reason, \$userId)**

MonthlyBillingRun

App\Billing\Services\MonthlyBillingRun

Corrida de facturación mensual de una sucursal.

- **__construct(\$overdueProcessor, \$invoiceGenerator)**
- **runForBranch(\$branchId, \$userId)**
Ejecuta la corrida para una sucursal.

OverdueProcessor

App\Billing\Services\OverdueProcessor

Procesador de mora.

- **markOverdueInvoices()**
Marca como Vencida toda factura pendiente con fecha de vencimiento anterior a hoy.
- **refreshContractSuspensions(\$branchId)**
Actualiza el conteo de facturas vencidas de los contratos de la sucursal y suspende los que alcanzan el umbral.

PaymentRegistrar

App\Billing\Services\PaymentRegistrar

Registro de pagos sobre facturas.

- **register(\$data, \$userId, \$branchId)**
Registra un pago validado sobre una factura.

Enumeraciones de facturación

InvoiceStatus

App\Billing\Enums\InvoiceStatus

Estados del ciclo de vida de una factura.

- **payable()**
Estados que admiten recibir pagos (tienen saldo exigible).
- **overdueCandidates()**
Estados que pasan a Vencida cuando se supera la fecha de vencimiento. PendienteRiesgoCorte se excluye deliberadamente: ya está en la ruta de suspensión y tiene su propia fecha de corte (comportamiento heredado que se preserva).
- **notPayable()**
Estados que ya no admiten pagos.
- **cases()**
- **from(\$value)**

- **tryFrom(\$value)**

InvoiceType

App\Billing\Enums\InvoiceType

Tipos de factura.

- **cases()**
- **from(\$value)**
- **tryFrom(\$value)**

PaymentStatus

App\Billing\Enums\PaymentStatus

Estados de un pago.

- **cases()**
- **from(\$value)**
- **tryFrom(\$value)**

ProrationMode

App\Billing\Enums\ProrationMode

Modo de facturación del primer mes de un contrato.

- **label()**
Etiqueta legible para formularios.
- **cases()**
- **from(\$value)**
- **tryFrom(\$value)**

Modelos

AdditionalCharge

App\Models\AdditionalCharge

Cargo adicional de un contrato.

- **user()**
- **contract()**
- **isDeferred()**
¿El cargo está diferido a cuotas? */
- **installmentAmount()**
Valor de la cuota regular (todas menos la última).
- **amountForInstallment(\$n)**
Valor de la cuota número \$n. La última absorbe la diferencia de redondeo para que la suma de cuotas sea exactamente el monto del cargo.
- **factory(\$count, \$state)**
Get a new factory instance for the model.

BillingRun

App\Models\BillingRun

Registro de una corrida de facturación.

- **branch()**
Sucursal de la corrida */
- **user()**
Usuario que ejecutó la generación */

BranchBillingSetting

App\Models\BranchBillingSetting

Configuración de facturación de una sucursal.

- **branch()**
Sucursal a la que pertenece la configuración */
- **forBranch(\$branchId)**
Configuración de una sucursal, creándola con los defaults si no existe todavía.
- **prorates()**
¿La sucursal prorratea el primer mes? */

Invoice

App\Models\Invoice

Modelo de Factura

- **user()**
Usuario que generó la factura */
- **branch()**
Sucursal a la que pertenece la factura */
- **invoice_items()**
Ítems (conceptos) de la factura */
- **contract()**
Contrato facturado */
- **payments()**
Pagos aplicados a la factura */
- **numberingSequence()**
Secuencia con la que se numeró la factura */
- **voidedBy()**
Usuario que anuló la factura */
- **displayNumber()**
Número para mostrar: el formal si existe, o el id para las facturas históricas anteriores a la numeración.
- **getPendingAmount()**
Saldo pendiente real: total menos los pagos completados. (pending_invoice_amount se recalcula con esta misma regla en Payment::updateInvoiceBalance; este método es la fuente de verdad al validar un pago.)
- **factory(\$count, \$state)**
Get a new factory instance for the model.
- **bootAuditable()**

- **writeAudit(\$action, \$old, \$new)**
Escribe el registro de auditoría del cambio.
- **audits()**
Historial de auditoría del modelo */

InvoiceItem

App\Models\InvoiceItem

- **invoice()**
- **factory(\$count, \$state)**
Get a new factory instance for the model.

InvoiceNumberingSequence

App\Models\InvoiceNumberingSequence

Secuencia de numeración de facturas de una sucursal.

- **branch()**
Sucursal dueña de la secuencia */
- **invoices()**
Facturas numeradas con esta secuencia */

Comandos de consola

NotifyInvoiceReminders

App\Console\Commands\NotifyInvoiceReminders

Envía los recordatorios de facturación al cliente:

- **handle()**

Trabajos en segundo plano

GeneratePendingInvoicesPdf

App\Jobs\GeneratePendingInvoicesPdf

- **__construct(\$branchId)**
- **handle()**
- **dispatch(\$arguments)**
Dispatch the job with the given arguments.
- **dispatchIf(\$boolean, \$arguments)**
Dispatch the job with the given arguments if the given truth test passes.
- **dispatchUnless(\$boolean, \$arguments)**
Dispatch the job with the given arguments unless the given truth test passes.
- **dispatchSync(\$arguments)**
Dispatch a command to its appropriate handler in the current process.
- **dispatchAfterResponse(\$arguments)**
Dispatch a command to its appropriate handler after the current process.
- **withChain(\$chain)**
Set the jobs that should run if this job is successful.

- **attempts()**
Get the number of times the job has been attempted.
- **delete()**
Delete the job from the queue.
- **fail(\$exception)**
Fail the job from the queue.
- **release(\$delay)**
Release the job back into the queue after (n) seconds.
- **setJob(\$job)**
Set the base queue job instance.
- **onConnection(\$connection)**
Set the desired connection for the job.
- **onQueue(\$queue)**
Set the desired queue for the job.
- **allOnConnection(\$connection)**
Set the desired connection for the chain.
- **allOnQueue(\$queue)**
Set the desired queue for the chain.
- **delay(\$delay)**
Set the desired delay in seconds for the job.
- **afterCommit()**
Indicate that the job should be dispatched after all database transactions have committed.
- **beforeCommit()**
Indicate that the job should not wait until database transactions have been committed before dispatching.
- **through(\$middleware)**
Specify the middleware the job should be dispatched through.
- **chain(\$chain)**
Set the jobs that should run if this job is successful.
- **prependToChain(\$job)**
Prepend a job to the current chain so that it is run after the currently running job.
- **appendToChain(\$job)**
Append a job to the end of the current chain.
- **dispatchNextJobInChain()**
Dispatch the next job on the chain.
- **invokeChainCatchCallbacks(\$e)**
Invoke all of the chain's failed job callbacks.
- **__serialize()**
Prepare the instance values for serialization.
- **__unserialize(\$values)**
Restore the model after serialization.
- **restoreModel(\$value)**

Restore the model from the model identifier instance.

Notificaciones

InvoiceDueSoon

App\Notifications\InvoiceDueSoon

Recordatorio al cliente unos días antes de que venza su factura.

- **__construct(\$invoice, \$diasRestantes)**
- **via(\$notifiable)**
- **toMail(\$notifiable)**
- **toWhatsApp(\$notifiable)**
- **onConnection(\$connection)**
Set the desired connection for the job.
- **onQueue(\$queue)**
Set the desired queue for the job.
- **allOnConnection(\$connection)**
Set the desired connection for the chain.
- **allOnQueue(\$queue)**
Set the desired queue for the chain.
- **delay(\$delay)**
Set the desired delay in seconds for the job.
- **afterCommit()**
Indicate that the job should be dispatched after all database transactions have committed.
- **beforeCommit()**
Indicate that the job should not wait until database transactions have been committed before dispatching.
- **through(\$middleware)**
Specify the middleware the job should be dispatched through.
- **chain(\$chain)**
Set the jobs that should run if this job is successful.
- **prependToChain(\$job)**
Prepend a job to the current chain so that it is run after the currently running job.
- **appendToChain(\$job)**
Append a job to the end of the current chain.
- **dispatchNextJobInChain()**
Dispatch the next job on the chain.
- **invokeChainCatchCallbacks(\$e)**
Invoke all of the chain's failed job callbacks.

InvoiceGenerated

App\Notifications\InvoiceGenerated

Aviso al cliente de que se generó una nueva factura.

- **__construct(\$invoice)**

- **via(\$notifiable)**
- **toMail(\$notifiable)**
- **toWhatsApp(\$notifiable)**
- **onConnection(\$connection)**
Set the desired connection for the job.
- **onQueue(\$queue)**
Set the desired queue for the job.
- **allOnConnection(\$connection)**
Set the desired connection for the chain.
- **allOnQueue(\$queue)**
Set the desired queue for the chain.
- **delay(\$delay)**
Set the desired delay in seconds for the job.
- **afterCommit()**
Indicate that the job should be dispatched after all database transactions have committed.
- **beforeCommit()**
Indicate that the job should not wait until database transactions have been committed before dispatching.
- **through(\$middleware)**
Specify the middleware the job should be dispatched through.
- **chain(\$chain)**
Set the jobs that should run if this job is successful.
- **prependToChain(\$job)**
Prepend a job to the current chain so that it is run after the currently running job.
- **appendToChain(\$job)**
Append a job to the end of the current chain.
- **dispatchNextJobInChain()**
Dispatch the next job on the chain.
- **invokeChainCatchCallbacks(\$e)**
Invoke all of the chain's failed job callbacks.

InvoiceOverdue

App\Notifications\InvoiceOverdue

Aviso al cliente de que su factura se venció.

- **__construct(\$invoice)**
- **via(\$notifiable)**
- **toMail(\$notifiable)**
- **toWhatsApp(\$notifiable)**
- **onConnection(\$connection)**
Set the desired connection for the job.
- **onQueue(\$queue)**
Set the desired queue for the job.

- **allOnConnection(\$connection)**
Set the desired connection for the chain.
- **allOnQueue(\$queue)**
Set the desired queue for the chain.
- **delay(\$delay)**
Set the desired delay in seconds for the job.
- **afterCommit()**
Indicate that the job should be dispatched after all database transactions have committed.
- **beforeCommit()**
Indicate that the job should not wait until database transactions have been committed before dispatching.
- **through(\$middleware)**
Specify the middleware the job should be dispatched through.
- **chain(\$chain)**
Set the jobs that should run if this job is successful.
- **prependToChain(\$job)**
Prepend a job to the current chain so that it is run after the currently running job.
- **appendToChain(\$job)**
Append a job to the end of the current chain.
- **dispatchNextJobInChain()**
Dispatch the next job on the chain.
- **invokeChainCatchCallbacks(\$e)**
Invoke all of the chain's failed job callbacks.

4.4 Pagos y cajas

Controladores

CashRegisterController

App\Http\Controllers\CashRegisterController

- **__construct()**
- **summary(\$request)**
Resumen/cuadre de cajas de la sucursal por período.
- **index()**
Display a listing of the resource.
- **create()**
Show the form for creating a new resource.
- **store(\$request)**
Store a newly created resource in storage.
- **show(\$cashRegister)**
Display the specified resource.
- **edit(\$cashRegister)**
Show the form for editing the specified resource.

- **update(\$request, \$cashRegister)**
Update the specified resource in storage.
- **destroy(\$cashRegister)**
Remove the specified resource from storage.
- **open(\$request)**
- **close(\$request)**
- **status()**
- **report(\$request)**

CashRegisterTransactionController

App\Http\Controllers\CashRegisterTransactionController

- **__construct()**
- **index()**
Display a listing of the resource.
- **history(\$request)**
- **exportHistoryTransactionsPDF(\$request)**
- **export()**
- **create()**
Show the form for creating a new resource.
- **store(\$request)**
Store a newly created resource in storage.
- **show(\$cashRegisterTransaction)**
Display the specified resource.
- **edit(\$cashRegisterTransaction)**
Show the form for editing the specified resource.
- **update(\$request, \$cashRegisterTransaction)**
Update the specified resource in storage.
- **destroy(\$cashRegisterTransaction)**
Remove the specified resource from storage.

PaymentController

App\Http\Controllers\PaymentController

Controlador de Pagos

- **__construct()**
Constructor: protege las rutas con autenticación y permisos.
- **index(\$request)**
Historial de pagos con filtros.
- **exportPaymentsPDF(\$request)**
Exporta el historial filtrado a PDF.
- **export()**
Exporta todos los pagos a Excel.
- **searchView(\$request)**

Vista de búsqueda de facturas para cobro. Si llegan parámetros de búsqueda (GET), ejecuta la búsqueda — así la paginación funciona con enlaces normales.

- **search(\$request)**
Búsqueda vía POST (compatibilidad con el formulario clásico).
- **store(\$request, \$registrar)**
Registra un pago sobre una factura.

Modelos

CashRegister

App\Models\CashRegister

- **transactions()**
- **user()**
- **branch()**
- **calculateTotals()**
- **forceDelete()**
Force a hard delete on a soft deleted model.
- **bootSoftDeletes()**
Boot the soft deleting trait for a model.
- **initializeSoftDeletes()**
Initialize the soft deleting trait for an instance.
- **forceDeleteQuietly()**
Force a hard delete on a soft deleted model without raising any events.
- **restore()**
Restore a soft-deleted model instance.
- **restoreQuietly()**
Restore a soft-deleted model instance without raising any events.
- **trashed()**
Determine if the model instance has been soft-deleted.
- **softDeleted(\$callback)**
Register a "softDeleted" model event callback with the dispatcher.
- **restoring(\$callback)**
Register a "restoring" model event callback with the dispatcher.
- **restored(\$callback)**
Register a "restored" model event callback with the dispatcher.
- **forceDeleting(\$callback)**
Register a "forceDeleting" model event callback with the dispatcher.
- **forceDeleted(\$callback)**
Register a "forceDeleted" model event callback with the dispatcher.
- **isForceDeleting()**
Determine if the model is currently force deleting.
- **getDeletedAtColumn()**
Get the name of the "deleted at" column.

- **getQualifiedDeletedAtColumn()**
Get the fully qualified "deleted at" column.

CashRegisterTransaction

App\Models\CashRegisterTransaction

- **cashRegister()**
Get the cash register associated with the transaction.
- **payment()**
Get the payment associated with the transaction.
- **user()**
Get the user who created the transaction.
- **forceDelete()**
Force a hard delete on a soft deleted model.
- **factory(\$count, \$state)**
Get a new factory instance for the model.
- **bootSoftDeletes()**
Boot the soft deleting trait for a model.
- **initializeSoftDeletes()**
Initialize the soft deleting trait for an instance.
- **forceDeleteQuietly()**
Force a hard delete on a soft deleted model without raising any events.
- **restore()**
Restore a soft-deleted model instance.
- **restoreQuietly()**
Restore a soft-deleted model instance without raising any events.
- **trashed()**
Determine if the model instance has been soft-deleted.
- **softDeleted(\$callback)**
Register a "softDeleted" model event callback with the dispatcher.
- **restoring(\$callback)**
Register a "restoring" model event callback with the dispatcher.
- **restored(\$callback)**
Register a "restored" model event callback with the dispatcher.
- **forceDeleting(\$callback)**
Register a "forceDeleting" model event callback with the dispatcher.
- **forceDeleted(\$callback)**
Register a "forceDeleted" model event callback with the dispatcher.
- **isForceDeleting()**
Determine if the model is currently force deleting.
- **getDeletedAtColumn()**
Get the name of the "deleted at" column.
- **getQualifiedDeletedAtColumn()**
Get the fully qualified "deleted at" column.

Payment

App\Models\Payment

Modelo de Pago

- **invoice()**
Factura sobre la que se aplicó el pago */
- **cashRegister()**
Caja registradora donde se recibió el pago (null si fue remoto) */
- **user()**
Usuario que registró el pago */
- **createAuditLog(\$action)**
Crea el registro de auditoría del pago.
- **updateInvoiceBalance()**
Recalcula el saldo pendiente de la factura asociada.
- **forceDelete()**
Force a hard delete on a soft deleted model.
- **bootSoftDeletes()**
Boot the soft deleting trait for a model.
- **initializeSoftDeletes()**
Initialize the soft deleting trait for an instance.
- **forceDeleteQuietly()**
Force a hard delete on a soft deleted model without raising any events.
- **restore()**
Restore a soft-deleted model instance.
- **restoreQuietly()**
Restore a soft-deleted model instance without raising any events.
- **trashed()**
Determine if the model instance has been soft-deleted.
- **softDeleted(\$callback)**
Register a "softDeleted" model event callback with the dispatcher.
- **restoring(\$callback)**
Register a "restoring" model event callback with the dispatcher.
- **restored(\$callback)**
Register a "restored" model event callback with the dispatcher.
- **forceDeleting(\$callback)**
Register a "forceDeleting" model event callback with the dispatcher.
- **forceDeleted(\$callback)**
Register a "forceDeleted" model event callback with the dispatcher.
- **isForceDeleting()**
Determine if the model is currently force deleting.
- **getDeletedAtColumn()**
Get the name of the "deleted at" column.
- **getQualifiedDeletedAtColumn()**

Get the fully qualified "deleted at" column.

PaymentAudit

App\Models\PaymentAudit

Modelo de Auditoría de Pagos

- **payment()**
Pago auditado */
- **user()**
Usuario que realizó la acción */
- **factory(\$count, \$state)**
Get a new factory instance for the model.

Exportaciones

PaymentsExport

App\Exports\PaymentsExport

- **query()**
- **headings()**
- **map(\$payment)**
- **download(\$fileName, \$writerType, \$headers)**
- **store(\$filePath, \$disk, \$writerType, \$diskOptions)**
- **queue(\$filePath, \$disk, \$writerType, \$diskOptions)**
- **raw(\$writerType)**
- **toResponse(\$request)**
Create an HTTP response that represents the object.

4.5 Órdenes técnicas

Controladores

TechnicalOrderController

App\Http\Controllers\TechnicalOrderController

Controlador de Órdenes Técnicas

- **__construct()**
Constructor: protege las rutas con autenticación y permisos.
- **index(\$request)**
Listado de órdenes técnicas de la sucursal con filtros.
- **orderVerification()**
Listado de órdenes Prefinalizadas pendientes de verificación por un supervisor.
- **verificationOrderProcess(\$request, \$technicalOrder)**
Procesa la verificación de una orden Prefinalizada.
- **orderReject(\$technicalOrder, \$request)**

El técnico rechaza una orden asignada (no puede ejecutarla). Queda en estado "Rechazada" con el motivo, visible para que oficina la reasigne o gestione.

- **create(\$contract)**
Formulario de creación de orden desde un contrato.
- **store(\$request)**
Crea una orden técnica para un contrato.
- **myTechnicalOrders()**
"Mis órdenes": listado de órdenes asignadas al técnico autenticado, junto con los materiales disponibles en su almacén personal para reportar al procesar.
- **getSerialNumbers(\$materialId)**
API JSON: seriales disponibles de un material en el almacén personal del técnico autenticado.
- **processOrder(\$request, \$id)**
El técnico procesa (ejecuta) una orden asignada.
- **show(\$technicalOrder)**
Detalle de una orden para su procesamiento por el técnico. Incluye los materiales disponibles en su almacén personal.
- **downloadPdf(\$technicalOrder)**
Comprobante en PDF de una orden técnica.
- **update(\$request, \$technicalOrder)**
Asigna (o reasigna) un técnico a la orden → estado "Asignada".
- **export(\$request)**
Exporta las órdenes a Excel (respetando los filtros del request, manejados dentro del export).

Modelos

TechnicalOrder

App\Models\TechnicalOrder

Modelo de Orden Técnica

- **contract()**
Contrato (cliente) al que pertenece el trabajo */
- **branch()**
Sucursal donde se ejecuta la orden */
- **assignedUser()**
Técnico asignado para ejecutar la orden */
- **createdBy()**
Usuario que creó la orden (oficina/suporte) */
- **materials()**
Materiales consumidos en la ejecución. Para equipos, cada registro incluye el serial instalado.
- **verifications()**
Verificaciones de supervisión posteriores al cierre */
- **factory(\$count, \$state)**
Get a new factory instance for the model.

TechnicalOrderMaterial

App\Models\TechnicalOrderMaterial

Modelo de Material de Orden Técnica

- **technicalOrder()**
Orden técnica en la que se usó el material */
- **material()**
Material del catálogo que se consumió */
- **factory(\$count, \$state)**
Get a new factory instance for the model.

TechnicalOrderVerification

App\Models\TechnicalOrderVerification

Modelo de Verificación de Orden Técnica

- **technicalOrder()**
Orden técnica verificada */
- **verifiedByUser()**
Usuario (supervisor) que realizó la verificación */
- **factory(\$count, \$state)**
Get a new factory instance for the model.

Notificaciones

TechnicalOrderAssignedTechnician

App\Notifications\TechnicalOrderAssignedTechnician

Aviso al técnico de que se le asignó una orden.

- **__construct(\$order)**
- **via(\$notifiable)**
- **toMail(\$notifiable)**
- **toWhatsApp(\$notifiable)**
- **toArray(\$notifiable)**
Datos que quedan en la tabla notifications: alimentan el contador y el aviso del navegador.
- **onConnection(\$connection)**
Set the desired connection for the job.
- **onQueue(\$queue)**
Set the desired queue for the job.
- **allOnConnection(\$connection)**
Set the desired connection for the chain.
- **allOnQueue(\$queue)**
Set the desired queue for the chain.
- **delay(\$delay)**
Set the desired delay in seconds for the job.
- **afterCommit()**
Indicate that the job should be dispatched after all database transactions have committed.

- **beforeCommit()**
Indicate that the job should not wait until database transactions have been committed before dispatching.
- **through(\$middleware)**
Specify the middleware the job should be dispatched through.
- **chain(\$chain)**
Set the jobs that should run if this job is successful.
- **prependToChain(\$job)**
Prepend a job to the current chain so that it is run after the currently running job.
- **appendToChain(\$job)**
Append a job to the end of the current chain.
- **dispatchNextJobInChain()**
Dispatch the next job on the chain.
- **invokeChainCatchCallbacks(\$e)**
Invoke all of the chain's failed job callbacks.

Informes

OrderDetailMap

App\Reports\Support\OrderDetailMap

Clasificación de las órdenes técnicas por su detalle.

- **clave(\$detalle)**
Normaliza un detalle a su clave de categoría.
- **etiqueta(\$clave)**
- **tipo(\$clave)**
- **color(\$clave)**
- **agrupar(\$conteos)**
Agrupa un conjunto de "detalle => cantidad" en categorías.

TechnicalOrdersReport

App\Reports\TechnicalOrdersReport

Operación de campo: volumen de órdenes técnicas y rendimiento de los técnicos.

- **__construct(\$period, \$branchId)**
- **series()**
Órdenes creadas y cerradas por período.
- **porTipo()**
Distribución por tipo de orden (Servicio / Incidencia).
- **porDetalle()**
Distribución por el detalle de la orden.
- **detallePorEstado()**
Detalle de las órdenes cruzado con su resultado.
- **porEstado()**
- **porTecnico()**

Rendimiento por técnico.

- **sinAsignar()**
Órdenes sin técnico asignado dentro del rango.
- **verificaciones()**
Resultado de las verificaciones de supervisión.
- **resumen()**
Indicadores del período con variación contra el anterior.

Exportaciones

TechnicalOrdersExport

App\Exports\TechnicalOrdersExport

- **__construct(\$request)**
- **query()**
- **headings()**
- **map(\$technical_order)**
- **download(\$fileName, \$writerType, \$headers)**
- **store(\$filePath, \$disk, \$writerType, \$diskOptions)**
- **queue(\$filePath, \$disk, \$writerType, \$diskOptions)**
- **raw(\$writerType)**
- **toResponse(\$request)**
Create an HTTP response that represents the object.

4.6 Inventario y almacenes

Controladores

CategoryController

App\Http\Controllers\CategoryController

- **__construct()**
- **index()**
Display a listing of the resource.
- **create()**
Show the form for creating a new resource.
- **store(\$request, \$category)**
Store a newly created resource in storage.
- **show(\$category)**
Display the specified resource.
- **edit(\$category)**
Show the form for editing the specified resource.
- **update(\$request, \$category)**
Update the specified resource in storage.
- **destroy(\$category)**

Remove the specified resource from storage.

MaterialController

App\Http\Controllers\MaterialController

Controlador de Materiales

- **__construct()**
Constructor: protege las rutas con autenticación y permisos.
- **index()**
Lista el catálogo de materiales.
- **create()**
Muestra el formulario de creación con las categorías disponibles.
- **store(\$request)**
Guarda un nuevo material en el catálogo.
- **edit(\$material)**
Muestra el formulario de edición de un material.
- **update(\$request, \$material)**
Actualiza un material del catálogo.
- **destroy(\$material)**
Elimina un material del catálogo.

MaterialMovementController

App\Http\Controllers\MaterialMovementController

Controlador de Movimientos de Material

- **__construct()**
Constructor: protege las rutas con autenticación y permisos.
- **index()**
Formulario de registro de movimientos.
- **store(\$request)**
Registra un movimiento de material (uno o varios materiales).
- **getAvailableSerialNumbers(\$warehouseId, \$materialId)**
API JSON: seriales disponibles de un material en un almacén. Alimenta el select de seriales del modal de registro.
- **getAvailableQuantity(\$warehouseId, \$materialId)**
API JSON: cantidad disponible de un material en un almacén. Equipos: número de filas (una por serial). Consumibles: suma de la columna quantity.
- **history(\$request)**
Historial de movimientos con filtros.
- **exportMovementsPDF(\$request)**
Exporta el historial filtrado a PDF. Usa exactamente los mismos filtros que history() vía applyFilters(), sin el límite del mes actual.
- **export()**
Exporta todos los movimientos a Excel.

WarehouseController

App\Http\Controllers\WarehouseController

Controlador de Almacenes (Warehouses)

- **__construct()**
Constructor: protege las rutas con autenticación y permisos.
- **index()**
Lista los almacenes de la sucursal activa.
- **show(\$warehouse)**
Muestra el detalle de un almacén con su inventario completo.
- **create()**
Muestra el formulario de creación de almacén.
- **store(\$request)**
Guarda un nuevo almacén.
- **edit(\$warehouse)**
Muestra el formulario de edición de un almacén.
- **update(\$request, \$warehouse)**
Actualiza un almacén existente.
- **destroy(\$warehouse)**
Elimina un almacén.
- **generatePdf(\$warehouse)**

Modelos

Category

App\Models\Category

- **materials()**
- **factory(\$count, \$state)**
Get a new factory instance for the model.

Inventory

App\Models\Inventory

- **warehouse()**
- **material()**
- **factory(\$count, \$state)**
Get a new factory instance for the model.

Material

App\Models\Material

Modelo de Material

- **category()**
Categoría a la que pertenece el material */
- **inventories()**

Registros de existencias del material en los almacenes. Para equipos, cada registro corresponde a un serial.

- **technicalOrders()**
Órdenes técnicas donde se usó este material */
- **factory(\$count, \$state)**
Get a new factory instance for the model.

MaterialMovement

App\Models\MaterialMovement

Modelo de Movimiento de Material

- **material()**
Material movido */
- **warehouseOrigin()**
Almacén de origen (null en entradas externas) */
- **warehouseDestination()**
Almacén de destino (null en salidas/bajas) */
- **user()**
Usuario que realizó el movimiento */
- **factory(\$count, \$state)**
Get a new factory instance for the model.

Warehouse

App\Models\Warehouse

Modelo de Almacén

- **user()**
Usuario que creó el almacén */
- **branch()**
Sucursal a la que pertenece el almacén */
- **inventories()**
Registros de inventario del almacén.
- **materials()**
Materiales del almacén, alcanzados a través del inventario.
- **factory(\$count, \$state)**
Get a new factory instance for the model.

Exportaciones

MaterialsMovementsExport

App\Exports\MaterialsMovementsExport

- **query()**
- **headings()**
- **map(\$movement)**
- **download(\$fileName, \$writerType, \$headers)**

- **store(\$filePath, \$disk, \$writerType, \$diskOptions)**
- **queue(\$filePath, \$disk, \$writerType, \$diskOptions)**
- **raw(\$writerType)**
- **toResponse(\$request)**

Create an HTTP response that represents the object.

4.7 Red GPON: OLT y ONT

Controladores

OltController

App\Http\Controllers\OltController

Controlador de OLTs

- **__construct(\$oltSshService)**
Constructor: inyecta el servicio SSH y protege las rutas con autenticación y permisos.
- **index()**
- **create()**
- **edit(\$olt)**
- **store(\$request)**
- **update(\$request, \$olt)**
Actualiza los datos de conexión de una OLT.
- **ontsAutofind(\$olt)**
- **apiOlt()**
- **viewVlans(\$olt)**
- **viewLineProfiles(\$olt)**
- **viewSrvProfiles(\$olt)**
- **storeVlan(\$request)**
Registra una VLAN de la OLT.
- **storeLineProfile(\$request)**
Registra un perfil de línea (line-profile) de la OLT.
- **storeSrvProfile(\$request)**
Registra un perfil de servicio (srv-profile) de la OLT.
- **updateVlan(\$request, \$vlan)**
- **updateLineProfile(\$request, \$lineProfile)**
- **updateSrvProfile(\$request, \$srvProfile)**
- **destroyVlan(\$vlan)**
- **destroyLineProfile(\$lineProfile)**
- **destroySrvProfile(\$srvProfile)**

OntController

App\Http\Controllers\OntController

Controlador de ONTs

- **__construct(\$oltSshService, \$snmpService, \$contractLinker)**
Constructor: inyecta los servicios SSH/SNMP y protege las rutas con autenticación y permisos.
- **no_authorized_ont_index()**
- **authorized_ont_index(\$request)**
Listado de ONTs autorizadas de la sucursal.
- **buscarContrato(\$request)**
- **linkContract(\$request, \$ont)**
Vincula la ONT a un contrato.
- **unlinkContract(\$ont)**
Quita la asociación de la ONT con su contrato.
- **activate(\$request)**
- **destroy(\$ont)**
- **syncPower(\$ont)**
- **checkSn(\$sn)**
- **relocate(\$request, \$ont)**
- **show(\$ont)**
- **realtimeInfo(\$request, \$ont)**
Endpoint AJAX: información en tiempo real de la ONT.
- **metricsHistory(\$request, \$ont)**
Endpoint AJAX: historial de métricas para las gráficas (potencia óptica y ancho de banda).
- **enableCatv(\$ont)**
- **disableCatv(\$ont)**
- **checkCatvState(\$ont)**
Consulta a la OLT el estado real del puerto CATV.
- **enableOnt(\$ont)**
Habilita la ONT (restablece el servicio del cliente).
- **disableOnt(\$ont)**
Deshabilita la ONT (corta el servicio sin borrar su configuración: se puede rehabilitar cuando se quiera).
- **reboot(\$ont)**
Reinicia la ONT del cliente.

OntImportController

App\Http\Controllers\OntImportController

Importación de ONTs existentes en una OLT.

- **__construct()**
- **index()**
- **preview(\$request, \$discovery)**
Análisis previo: consulta la OLT y responde qué se importaría, SIN escribir nada en la base de datos.
- **store(\$request)**
Encola la importación.

- **status(\$run)**
Estado de una corrida, para la barra de progreso.

Servicios

OltOntDiscovery

App\Services\OltOntDiscovery

Descubrimiento del inventario de ONTs de una OLT.

- **__construct(\$snmp)**
- **discover(\$olt)**
Lee de la OLT todas las ONTs registradas.
- **decodeSerial(\$raw)**
Convierte el serial binario de Huawei al formato legible.
- **matchContract(\$description, \$branchId)**
Intenta identificar el contrato al que pertenece una ONT a partir de su descripción en la OLT.
- **preview(\$olt, \$limit)**
Resumen de lo que traería una importación, sin escribir nada.

OltSnmpService

App\Services\OltSnmpService

Consulta de métricas de ONTs por SNMP.

- **getOntMetrics(\$olt, \$ont, \$useCache)**
Métricas en vivo de una ONT.
- **bulkOntMetrics(\$olt, \$onts)**
Métricas de TODAS las ONTs indicadas, en pocas peticiones.
- **bulkTrafficCounters(\$olt)**
Contadores de tráfico (octetos in/out) de las ONTs que tengan su ifIndex propio resuelto.
- **interfaceDescriptions(\$olt)**
Recorre ifDescr y devuelve el mapa de descripciones de interfaz. Se usa para resolver ifIndex de puertos PON y de ONTs, y es lo que muestra el comando de diagnóstico.
- **resolvePonPortIfIndex(\$olt, \$slot, \$port)**
ifIndex del PUERTO PON al que pertenece una ONT. (Reemplaza al resolveIfIndex que estaba en el controlador.)
- **resolveOntIfIndex(\$olt, \$ont, \$descriptions)**
ifIndex PROPIO de la ONT, necesario para las estadísticas de tráfico (es distinto del ifIndex del puerto PON, que agrupa a todas las ONTs de ese puerto).
- **metricDefinitions(\$olt)**
Definiciones de métricas de la marca de la OLT.
- **syncRxPower(\$olt)**
Compatibilidad: actualiza la potencia de todas las ONTs de una OLT. El comando onts:sync-power lo sigue usando; internamente ya no hace SNMPv1 sino recorridos masivos v2c.
- **syncSingleOntPower(\$olt, \$ont)**
Compatibilidad: refresca la potencia de una ONT concreta.

OltSshService

App\Services\OltSshService

- **getOltStatus(\$olt)**
Obtiene el estado de la OLT (temperatura, uptime, etc.)
- **getAutoFindOnt(\$olt)**
Obtiene las ONTs en modo autofind
- **activateOnt(\$olt, \$data)**
Activa una ONT en la OLT via SSH y retorna el ont_id asignado
- **resolveServicePort(\$olt, \$ont)**
Consulta a la OLT el service-port de una ONT ya existente.
- **deleteOnt(\$olt, \$ont)**
Elimina una ONT de la OLT via SSH
- **moveOnt(\$olt, \$ont, \$newData)**
Mueve una ONT de un puerto a otro en la misma OLT 1. Elimina service-port y ONT del puerto viejo 2. Activa la ONT en el puerto nuevo Retorna array con ont_id y service_port nuevos
- **getCatvPortState(\$olt, \$ont)**
Consulta el estado del puerto CATV de una ONT Retorna 'on', 'off' o null si no se pudo determinar
- **getOntOpticalInfo(\$olt, \$ont)**
- **setCatvPort(\$olt, \$ont, \$enable)**
Habilita o deshabilita el puerto CATV de una ONT
- **setOntAdminState(\$olt, \$ont, \$enable)**
Habilita o deshabilita la ONT completa en la OLT.
- **rebootOnt(\$olt, \$ont)**
Reinicia la ONT del cliente.

OntPoller

App\Services\OntPoller

Muestreo masivo de métricas de ONTs.

- **__construct(\$snmp)**
- **poll(\$olt, \$resolveTrafficIndexes)**
Muestrea todas las ONTs de una OLT.
- **pruneOldMetrics(\$days)**
Elimina muestras antiguas para que el historial no crezca sin límite. Por defecto conserva 30 días.

Modelos

LineProfile

App\Models\LineProfile

- **olt()**
- **factory(\$count, \$state)**
Get a new factory instance for the model.

Olt

App\Models\Olt

- **onts()**
Relación con ONTs
- **branch()**
Relación con Branch
- **lineProfiles()**
- **srvProfiles()**
- **vlangs()**
- **getPlainPassword()**
Obtiene la contraseña sin cifrar para conexiones SSH NOTA: Solo usar si la contraseña se guarda en texto plano
- **isActive()**
Verifica si la OLT está activa
- **isConnected()**
Verifica si la OLT está conectada según el último estado
- **getStatusTextAttribute()**
Obtiene el texto del estado de conexión
- **scopeActive(\$query)**
Scope para obtener solo OLTs activas
- **scopeByBranch(\$query, \$branchId)**
Scope para obtener OLTs por branch
- **factory(\$count, \$state)**
Get a new factory instance for the model.

Ont

App\Models\Ont

ONT: el equipo de fibra instalado en casa del cliente.

- **branch()**
- **olt()**
- **contract()**
- **metrics()**
Historial de métricas (muestras del poller) */
- **latestMetric()**
Última muestra registrada */
- **factory(\$count, \$state)**
Get a new factory instance for the model.

OntImportRun

App\Models\OntImportRun

Corrida de importación de ONTs desde una OLT.

- **olt()**
- **user()**

- **enCurso()**
¿La corrida sigue en curso? */
- **porcentaje()**
Porcentaje de avance para la barra de progreso */
- **estadoLegible()**
Texto del estado para mostrar al usuario */

OntMetric

App\Models\OntMetric

Muestra histórica de las métricas de una ONT.

- **ont()**

SrvProfile

App\Models\SrvProfile

- **olt()**
- **factory(\$count, \$state)**
Get a new factory instance for the model.

VlanOlt

App\Models\VlanOlt

- **olt()**
- **factory(\$count, \$state)**
Get a new factory instance for the model.

Comandos de consola

ImportOnt

App\Console\Commands\ImportOnt

Importa por consola las ONTs existentes en una OLT.

- **handle(\$discovery)**

OltSnmpProbe

App\Console\Commands\OltSnmpProbe

Diagnóstico SNMP de una OLT.

- **__construct(\$snmp)**
- **handle()**

PollOnt

App\Console\Commands\PollOnt

Muestrea por SNMP las métricas de todas las ONTs.

- **__construct(\$poller)**
- **handle()**

SyncOltInterfaces

App\Console\Commands\SyncOltInterfaces

Mapea las interfaces de la OLT y actualiza los índices SNMP de las ONTs.

- **__construct(\$snmp)**
- **handle()**

SyncOntPower

App\Console\Commands\SyncOntPower

- **__construct(\$snmpService)**
- **handle()**

Trabajos en segundo plano

ImportOltOnt

App\Jobs\ImportOltOnt

Importa a GestISP las ONTs que ya existen en una OLT.

- **__construct(\$runId)**
- **handle(\$discovery)**
- **failed(\$e)**
Si el job muere por una excepción no controlada, la corrida no debe quedar "en ejecución" para siempre.
- **dispatch(\$arguments)**
Dispatch the job with the given arguments.
- **dispatchIf(\$boolean, \$arguments)**
Dispatch the job with the given arguments if the given truth test passes.
- **dispatchUnless(\$boolean, \$arguments)**
Dispatch the job with the given arguments unless the given truth test passes.
- **dispatchSync(\$arguments)**
Dispatch a command to its appropriate handler in the current process.
- **dispatchAfterResponse(\$arguments)**
Dispatch a command to its appropriate handler after the current process.
- **withChain(\$chain)**
Set the jobs that should run if this job is successful.
- **attempts()**
Get the number of times the job has been attempted.
- **delete()**
Delete the job from the queue.
- **fail(\$exception)**
Fail the job from the queue.
- **release(\$delay)**
Release the job back into the queue after (n) seconds.
- **setJob(\$job)**
Set the base queue job instance.
- **onConnection(\$connection)**

Set the desired connection for the job.

- **onQueue(\$queue)**
Set the desired queue for the job.
- **allOnConnection(\$connection)**
Set the desired connection for the chain.
- **allOnQueue(\$queue)**
Set the desired queue for the chain.
- **delay(\$delay)**
Set the desired delay in seconds for the job.
- **afterCommit()**
Indicate that the job should be dispatched after all database transactions have committed.
- **beforeCommit()**
Indicate that the job should not wait until database transactions have been committed before dispatching.
- **through(\$middleware)**
Specify the middleware the job should be dispatched through.
- **chain(\$chain)**
Set the jobs that should run if this job is successful.
- **prependToChain(\$job)**
Prepend a job to the current chain so that it is run after the currently running job.
- **appendToChain(\$job)**
Append a job to the end of the current chain.
- **dispatchNextJobInChain()**
Dispatch the next job on the chain.
- **invokeChainCatchCallbacks(\$e)**
Invoke all of the chain's failed job callbacks.
- **__serialize()**
Prepare the instance values for serialization.
- **__unserialize(\$values)**
Restore the model after serialization.
- **restoreModel(\$value)**
Restore the model from the model identifier instance.

4.8 Red PPPoE: routers MikroTik

Controladores

PppoeAccountController

App\Http\Controllers\PppoeAccountController

Controlador de cuentas PPPoE

- **__construct(\$mikrotik, \$contractLinker)**
Constructor: inyecta el servicio Mikrotik y protege las rutas con autenticación y permisos.
- **index()**

- **store(\$request)**
- **update(\$request, \$pppoe)**
- **linkContract(\$request, \$pppoe)**
Vincula la cuenta a un contrato.
- **unlinkContract(\$pppoe)**
- **toggleState(\$pppoe)**
- **destroy(\$pppoe)**
- **apiActiveSessions(\$router)**
API JSON — sesiones activas de un router
- **importFromRouter(\$router)**
Importa a la DB los secrets que ya existen en el router
- **show(\$pppoe)**
- **realtimeSession(\$pppoe)**
Endpoint AJAX: estado de conexión en tiempo real
- **metricsHistory(\$request, \$pppoe)**
Endpoint AJAX: historial de tráfico para la gráfica de ancho de banda. Las muestras las genera el comando pppoe:poll.
- **restartSession(\$pppoe)**
Reinicia la sesión del usuario (tumba la conexión activa)

RouterController

App\Http\Controllers\RouterController

Controlador de Routers (Mikrotik)

- **__construct(\$mikrotik)**
Constructor: inyecta el servicio Mikrotik y protege las rutas con autenticación y permisos.
- **index()**
- **create()**
- **edit(\$router)**
- **store(\$request)**
- **update(\$request, \$router)**
- **destroy(\$router)**
- **apiRouters()**
API JSON — routers con estado en tiempo real (para el DataTable)
- **apiProfiles(\$router)**
API JSON — perfiles PPP de un router (para selects)

Servicios

MikrotikApiService

App\Services\MikrotikApiService

- **getSystemInfo(\$router)**
Obtiene información del sistema (identity, version, uptime)

- **getPppProfiles(\$router)**
Lista los perfiles PPP disponibles en el router
- **getPppSecrets(\$router)**
Lista los secrets (cuentas pppoe) del router
- **createPppSecret(\$router, \$data)**
Crea una cuenta pppoe (secret) en el router Retorna el .id de mikrotik
- **updatePppSecret(\$router, \$account, \$data)**
Actualiza una cuenta pppoe existente
- **setPppSecretState(\$router, \$account, \$disabled)**
Habilita o deshabilita una cuenta pppoe
- **deletePppSecret(\$router, \$account)**
Elimina una cuenta pppoe del router
- **getActiveSessions(\$router)**
Lista las sesiones pppoe activas
- **getActiveSession(\$router, \$username)**
Obtiene la sesión activa de un usuario específico Retorna null si no está conectado
- **getPppoeInterfaceCounters(\$router)**
Contadores de tráfico de TODAS las interfaces del router.
- **getSessionTraffic(\$router, \$username)**
Velocidad instantánea de una sesión PPPoE.
- **dropActiveSession(\$router, \$username)**
Tumba la sesión activa de un usuario

Modelos

PppoeAccount

App\Models\PppoeAccount

- **branch()**
- **router()**
- **contract()**
- **metrics()**
Historial de tráfico (muestras del poller) */
- **latestMetric()**
Última muestra registrada */
- **factory(\$count, \$state)**
Get a new factory instance for the model.

PppoeSessionMetric

App\Models\PppoeSessionMetric

Muestra histórica del tráfico de una cuenta PPPoE.

- **account()**

Router

App\Models\Router

- **branch()**
- **pppoeAccounts()**
- **getPlainPassword()**
- **scopeByBranch(\$query, \$branchId)**
- **scopeActive(\$query)**
- **factory(\$count, \$state)**
Get a new factory instance for the model.

4.9 Notificaciones

Controladores

NotificationController

App\Http\Controllers\NotificationController

Notificaciones en pantalla del usuario autenticado.

- **__construct()**
- **poll(\$request)**
Estado actual para el sondeo del navegador.
- **markAllRead(\$request)**
Marca todas las notificaciones del usuario como leídas.

Notificaciones

LogGateway

App\Notifications\WhatsApp\LogGateway

Proveedor simulado de WhatsApp.

- **send(\$to, \$message)**

MetaCloudGateway

App\Notifications\WhatsApp\MetaCloudGateway

Proveedor WhatsApp Cloud API oficial de Meta.

- **send(\$to, \$message)**

RespetaCanales

App\Notifications\Concerns\RespetaCanales

Arma la lista de canales de una notificación respetando los interruptores de configuración y lo que el destinatario tiene disponible.

Sin métodos públicos propios.

WhatsAppChannel

App\Notifications\Channels\WhatsAppChannel

Canal de notificaciones "whatsapp".

- **__construct(\$gateway)**
- **send(\$notifiable, \$notification)**

WhatsAppGateway

App\Notifications\WhatsApp\WhatsAppGateway

Contrato de un proveedor de envío de WhatsApp.

- **send(\$to, \$message)**
Envía un mensaje a un número ya normalizado (E.164 sin "+").

WhatsAppMessage

App\Notifications\Messages\WhatsAppMessage

Mensaje de WhatsApp que produce una notificación.

- **__construct(\$body, \$templateName)**
- **make(\$body)**
- **body(\$body)**
Texto del mensaje (para driver simulado y para ventana abierta).
- **template(\$name, \$params)**
Plantilla aprobada de Meta y sus variables, en orden.
- **templateLanguage(\$language)**
Define el idioma de esta plantilla sin afectar el de las notificaciones operativas restantes.

Utilidades

PhoneNumber

App\Support\PhoneNumber

Normaliza números de teléfono al formato que exige WhatsApp.

- **forWhatsApp(\$raw, \$countryCode)**
Devuelve el número listo para WhatsApp, o null si no parece un número válido (menos de 10 dígitos tras limpiar).

4.10 Informes gerenciales

Controladores

ManagementReportController

App\Http\Controllers\ManagementReportController

Informes gerenciales.

- **__construct()**
- **index(\$request)**
Tablero ejecutivo: los indicadores de los tres informes en una sola pantalla.
- **crecimiento(\$request)**
- **tecnicas(\$request)**
- **facturacion(\$request)**

- **aprovisionamiento(\$request)**
- **aprovisionamientoPdf(\$request)**
- **resumenPdf(\$request)**
- **crecimientoPdf(\$request)**
- **tecnicasPdf(\$request)**
- **facturacionPdf(\$request)**

Informes

BillingReport

App\Reports\BillingReport

Facturación, recaudo y cartera.

- **__construct(\$period, \$branchId)**
- **series()**
Facturado y recaudado por período.
- **carteraPorAntigüedad()**
Cartera pendiente agrupada por antigüedad del vencimiento.
- **porMetodoPago()**
Recaudo por método de pago.
- **facturasPorEstado()**
Facturas emitidas en el período, por estado.
- **mayoresDeudores(\$limite)**
Contratos con mayor saldo pendiente.
- **resumen()**
Indicadores del período con comparación contra el anterior.

Granularity

App\Reports\Enums\Granularity

Granularidad temporal de un informe gerencial.

- **label()**
- **sqlFormat()**
Formato de DATE_FORMAT con el que MySQL agrupa.
- **phpFormat()**
Formato de PHP equivalente al de sqlFormat().
- **humanize(\$bucket)**
Etiqueta legible de una clave de período.
- **maxDays()**
Rango máximo razonable, en días, para esta granularidad.
- **fromRequest(\$valor)**
- **cases()**
- **from(\$value)**
- **tryFrom(\$value)**

GrowthReport

App\Reports\GrowthReport

Crecimiento de la base de contratos.

- **__construct(\$period, \$branchId)**
- **series()**
Serie de altas, bajas y neto por período.
- **distribucionEstados()**
Distribución actual de la base por estado canónico.
- **porPlan()**
Contratos e ingreso recurrente por plan.
- **resumen()**
Indicadores del período, con variación contra el anterior.

ProvisioningReport

App\Reports\ProvisioningReport

Aprovisionamiento de red.

- **__construct(\$period, \$branchId)**
- **resumen()**
Indicadores del estado actual de la red.
- **cobertura()**
Contratos vigentes cruzados con sus equipos.
- **series()**
Altas de ONTs y cuentas PPPoE por período.
- **porOlt()**
ONTs por OLT, con su ocupación y estado.
- **porRouter()**
Cuentas PPPoE por router.
- **porPerfil()**
Cuentas PPPoE por perfil de velocidad.
- **calidadOptica()**
Distribución de la potencia óptica recibida por las ONTs.
- **ontsHuerfanas(\$limite)**
ONTs registradas sin contrato asociado.

ReportPeriod

App\Reports\Support\ReportPeriod

Período analizado por un informe gerencial.

- **__construct(\$from, \$to, \$granularity)**
- **fromRequest(\$desde, \$hasta, \$granularidad)**
Construye el período a partir de los filtros de la pantalla.
- **sqlBucket(\$columna)**
Expresión SQL que agrupa una columna de fecha por período.

- **buckets()**
Todas las claves de período del rango, en orden y sin huecos.
- **completarSerie(\$datos, \$relleno)**
Rellena los períodos sin datos.
- **etiquetas()**
Etiquetas legibles de la serie, para el eje de las gráficas.
- **anterior()**
Período inmediatamente anterior, de la misma duración.
- **demasiadoLargo()**
¿El rango es demasiado largo para esta granularidad?
- **granularidadSugerida()**
- **etiquetaRango()**

4.11 Usuarios, roles y permisos

Controladores

AdminController

App\Http\Controllers\AdminController

- **__construct()**
Constructor: protege el dashboard con autenticación y el permiso gestisp.index (todos los roles del seeder lo tienen).
- **index()**

BranchController

App\Http\Controllers/BranchController

Controlador de Sucursales (Branches)

- **__construct()**
Constructor: protege las rutas con autenticación y permisos.
- **index()**
Lista todas las sucursales.
- **create()**
Muestra el formulario de creación de sucursal.
- **store(\$request)**
Guarda una nueva sucursal en la base de datos.
- **show(\$branch)**
Muestra el detalle de una sucursal específica.
- **edit(\$branch)**
Muestra el formulario de edición de una sucursal.
- **update(\$request, \$branch)**
Actualiza los datos de una sucursal existente.
- **destroy(\$branch)**
Elimina una sucursal.

RoleController

App\Http\Controllers\RoleController

Controlador de Roles

- **__construct()**
Constructor: protege las rutas con autenticación y permisos. Antes este controlador no tenía protección y cualquier usuario autenticado podía gestionar roles.
- **index()**
Lista los roles del sistema.
- **create()**
Muestra el formulario de creación con los permisos agrupados por módulo.
- **store(\$request)**
Guarda un nuevo rol con sus permisos.
- **show(\$role)**
Display the specified resource.
- **edit(\$role)**
Muestra el formulario de edición con los permisos agrupados por módulo y los ids de los permisos que el rol ya tiene.
- **update(\$request, \$role)**
Actualiza el nombre y los permisos del rol.
- **destroy(\$role)**
Elimina un rol.

UserController

App\Http\Controllers\UserController

Controlador de Usuarios

- **__construct()**
Constructor: protege las rutas con autenticación y permisos.
- **index()**
Lista los usuarios del sistema.
- **create()**
Muestra el formulario de creación con sucursales y roles disponibles.
- **store(\$request)**
Guarda un nuevo usuario con sus asignaciones de sucursal/rol.
- **show(\$user, \$request)**
Trazabilidad del usuario: sesiones activas, historial, estadísticas de acceso e intentos fallidos.
- **closeSession(\$user, \$session)**
Cierra de forma remota una sesión activa del usuario.
- **toggleActive(\$user)**
Habilita o inhabilita el acceso del usuario.
- **closeSessions(\$user, \$request)**
Cierra de forma remota todas las sesiones activas del usuario.
- **edit(\$user)**

Muestra el formulario de edición con las asignaciones actuales.

- **update(\$request, \$user)**
Actualiza los datos del usuario y sus asignaciones.
- **destroy(\$user)**
Elimina un usuario del sistema.

Modelos

Branch

App\Models\Branch

- **users()**
- **clients()**
- **services()**
- **plans()**
- **contracts()**
- **cashRegister()**
- **warehouses()**
- **olts()**
- **onts()**
- **billingSettings()**
Configuración de facturación de la sucursal. Usar BranchBillingSetting::forBranch() cuando se necesite con defaults garantizados.
- **factory(\$count, \$state)**
Get a new factory instance for the model.

User

App\Models\User

- **branches()**
- **getFullNameAttribute()**
Nombre y apellido, para no repetir la concatenación. */
- **getInitialsAttribute()**
Iniciales del usuario, para el avatar de respaldo.
- **adminlte_image()**
Foto de perfil para el menú superior.
- **adminlte_desc()**
Texto bajo el nombre en el menú de usuario: el rol activo y la sucursal en la que está trabajando ahora mismo.
- **adminlte_profile_url()**
Destino del botón "Perfil" del menú de usuario. */
- **clients()**
- **services()**
- **plans()**

- **contracts()**
- **payments()**
- **additionalCharges()**
- **invoices()**
- **cashRegisters()**
- **cashRegisterTransactions()**
- **warehouses()**
- **materialMovements()**
- **technicalOrders()**
- **assignedTechnicalOrders()**
- **technicalOrderVerifications()**
- **routeNotificationForWhatsApp()**
Número al que se le envía WhatsApp (para los avisos a técnicos).
- **sessions()**
- **failedLogins()**
- **assignBranch(\$branchId, \$roleId)**
- **getCurrentRole()**
- **can(\$permission, \$arguments)**
- **tokens()**
Get the access tokens that belong to model.
- **tokenCan(\$ability)**
Determine if the current API token has a given scope.
- **createToken(\$name, \$abilities, \$expiresAt)**
Create a new personal access token for the user.
- **generateTokenString()**
Generate the token string.
- **currentAccessToken()**
Get the access token currently associated with the user.
- **withAccessToken(\$accessToken)**
Set the current access token for the user.
- **factory(\$count, \$state)**
Get a new factory instance for the model.
- **notifications()**
Get the entity's notifications.
- **readNotifications()**
Get the entity's read notifications.
- **unreadNotifications()**
Get the entity's unread notifications.
- **notify(\$instance)**
Send the given notification.

- **notifyNow(\$instance, \$channels)**
Send the given notification immediately.
- **routeNotificationFor(\$driver, \$notification)**
Get the notification routing information for the given driver.
- **bootHasRoles()**
- **getRoleClass()**
- **roles()**
A model may have multiple roles.
- **scopeRole(\$query, \$roles, \$guard, \$without)**
Scope the model query to certain roles only.
- **scopeWithoutRole(\$query, \$roles, \$guard)**
Scope the model query to only those without certain roles.
- **assignRole(\$roles)**
Assign the given role to the model.
- **removeRole(\$role)**
Revoke the given role from the model.
- **syncRoles(\$roles)**
Remove all current roles and set the given ones.
- **hasRole(\$roles, \$guard)**
Determine if the model has (one of) the given role(s).
- **hasAnyRole(\$roles)**
Determine if the model has any of the given role(s).
- **hasAllRoles(\$roles, \$guard)**
Determine if the model has all of the given role(s).
- **hasExactRoles(\$roles, \$guard)**
Determine if the model has exactly all of the given role(s).
- **getDirectPermissions()**
Return all permissions directly coupled to the model.
- **getRoleNames()**
- **bootHasPermissions()**
- **getPermissionClass()**
- **getWildcardClass()**
- **permissions()**
A model may have multiple direct permissions.
- **scopePermission(\$query, \$permissions, \$without)**
Scope the model query to certain permissions only.
- **scopeWithoutPermission(\$query, \$permissions)**
Scope the model query to only those without certain permissions, whether indirectly by role or by direct permission.
- **filterPermission(\$permission, \$guardName)**
Find a permission.
- **hasPermissionTo(\$permission, \$guardName)**

Determine if the model may perform the given permission.

- **checkPermissionTo(\$permission, \$guardName)**
An alias to hasPermissionTo(), but avoids throwing an exception.
- **hasAnyPermission(\$permissions)**
Determine if the model has any of the given permissions.
- **hasAllPermissions(\$permissions)**
Determine if the model has all of the given permissions.
- **hasDirectPermission(\$permission)**
Determine if the model has the given permission.
- **getPermissionsViaRoles()**
Return all the permissions the model has via roles.
- **getAllPermissions()**
Return all the permissions the model has, both directly and via roles.
- **givePermissionTo(\$permissions)**
Grant the given permission(s) to a role.
- **forgetWildcardPermissionIndex()**
- **syncPermissions(\$permissions)**
Remove all current permissions and set the given ones.
- **revokePermissionTo(\$permission)**
Revoke the given permission(s).
- **getPermissionNames()**
- **forgetCachedPermissions()**
Forget the cached permissions.
- **hasAllDirectPermissions(\$permissions)**
Check if the model has All of the requested Direct permissions.
- **hasAnyDirectPermission(\$permissions)**
Check if the model has Any of the requested Direct permissions.

Comandos de consola

SyncPermissions

App\Console\Commands\SyncPermissions

Sincroniza los permisos declarados en el código con la base de datos.

- **handle()**

Utilidades

PermissionLabels

App\Support\PermissionLabels

Etiquetas legibles de permisos.

- **module(\$permission)**
Nombre del módulo al que pertenece un permiso.
- **describe(\$permission)**

Descripción legible de un permiso, para mostrarla en el formulario de roles: "Ver almacenes", "Eliminar facturas".

Filtros

RoleBasedMenuFilter

App\Filters\RoleBasedMenuFilter

Filtro de menú por rol activo de la sesión.

- **transform(\$item)**
Marca con 'restricted' los ítems cuyo permiso no tiene el rol activo; AdminLTE los excluye del menú compilado (y poda automáticamente los padres cuyo submenú queda vacío).

4.12 Trazabilidad y auditoría

Controladores

AuditController

App\Http\Controllers\AuditController

Trazabilidad del sistema (solo superadministrador).

- **__construct()**
- **index(\$request)**
Bitácora con filtros.
- **show(\$audit)**
Detalle de un registro: contexto completo y campos que cambiaron.

Servicios

AuditLabels

App\Services\Audit\AuditLabels

Traduce lo técnico a lo que entiende una persona.

- **modelo(\$clase)**
Nombre legible del modelo. */
- **categoriaDeModelo(\$clase)**
Módulo al que pertenece el modelo. */
- **identificar(\$model)**
Cómo identificar el registro concreto: se busca el campo más reconocible (nombre, serial, número) y si no hay, el id.
- **categorias()**
Nombre legible de los módulos, para los filtros de la pantalla.
- **accion(\$action)**
Nombre legible de las acciones.

AuditLogger

App\Services\Audit\AuditLogger

Punto único por donde pasa TODO lo que se registra en la auditoría.

- **model(\$model, \$action, \$old, \$new)**
Registra el cambio de un modelo.
- **action(\$action, \$description, \$context, \$subject, \$category)**
Registra una acción que no es un cambio de modelo: reiniciar una ONT, exportar un listado, iniciar sesión, generar un PDF...
- **requestId()**
Identificador de la petición en curso (se crea la primera vez).
- **resetRequestId()**
Reinicia el identificador (lo usan las pruebas). */

Modelos

Audit

App\Models\Audit

Registro de trazabilidad del sistema.

- **auditable()**
Modelo auditado (si la acción afectó a uno) */
- **user()**
Usuario que realizó la acción (null en procesos de consola) */
- **branch()**
Sucursal en la que se estaba trabajando */
- **getAccionLegibleAttribute()**
Nombre legible de la acción */
- **getCategoriaLegibleAttribute()**
Nombre legible del módulo */
- **getCambiosAttribute()**
Campos que cambiaron, con su valor anterior y el nuevo.
- **getFalloAttribute()**
¿La acción falló? */
- **scopeDelUsuario(\$query, \$userId)**
- **scopeDeCategoria(\$query, \$categoria)**
- **scopeEntreFechas(\$query, \$desde, \$hasta)**

Comandos de consola

PruneAudits

App\Console\Commands\PruneAudits

Depura la bitácora de trazabilidad.

- **handle()**

Middleware

AuditRequests

App\Http\Middleware\AuditRequests

Registra cada acción que un usuario ejecuta en el sistema.

- **handle(\$request, \$next)**

Proveedores de servicios

AuditServiceProvider

App\Providers\AuditServiceProvider

Enchufa la trazabilidad a TODO el sistema.

- **boot()**

4.13 Infraestructura y utilidades

Controladores

Controller

App\Http\Controllers\Controller

- **authorize(\$ability, \$arguments)**
Authorize a given action for the current user.
- **authorizeForUser(\$user, \$ability, \$arguments)**
Authorize a given action for a user.
- **authorizeResource(\$model, \$parameter, \$options, \$request)**
Authorize a resource action based on the incoming request.
- **validateWith(\$validator, \$request)**
Run the validation routine against the given validator.
- **validate(\$request, \$rules, \$messages, \$attributes)**
Validate the given request with the given rules.
- **validateWithBag(\$errorBag, \$request, \$rules, \$messages, \$attributes)**
Validate the given request with the given rules.

HomeController

App\Http\Controllers\HomeController

- **__construct()**
Create a new controller instance.
- **index()**
Show the application dashboard.

PlanController

App\Http\Controllers\PlanController

Controlador de Planes

- **__construct()**
Constructor: protege las rutas con autenticación y permisos.
- **index()**
Lista los planes de la sucursal activa.
- **create()**

Muestra el formulario de creación de plan con los servicios de la sucursal disponibles para asociar.

- **store(\$request)**
Guarda un nuevo plan y asocia sus servicios.
- **edit(\$plan)**
Muestra el formulario de edición de un plan.
- **update(\$request, \$plan)**
Actualiza un plan y sincroniza sus servicios.
- **destroy(\$plan)**
Elimina un plan.

ServiceController

App\Http\Controllers\ServiceController

Controlador de Servicios

- **__construct()**
Constructor: protege las rutas con autenticación y permisos.
- **index()**
Lista los servicios de la sucursal activa.
- **create()**
Muestra el formulario de creación de servicio.
- **store(\$request)**
Guarda un nuevo servicio.
- **edit(\$service)**
Muestra el formulario de edición de un servicio.
- **update(\$request, \$service)**
Actualiza un servicio existente.
- **destroy(\$service)**
Elimina un servicio.

Servicios

PppoePoller

App\Services\PppoePoller

Muestreo del tráfico de las cuentas PPPoE.

- **__construct(\$mikrotik)**
- **poll(\$router)**
Muestrea todas las cuentas de un router.
- **pruneOldMetrics(\$days)**
Elimina muestras antiguas para acotar el historial.

Modelos

PdfReport

App\Models\PdfReport

- **factory(\$count, \$state)**

Get a new factory instance for the model.

Plan

App\Models\Plan

- **user()**
- **services()**
- **contracts()**
- **branch()**
- **assignService(\$serviceName)**
- **factory(\$count, \$state)**

Get a new factory instance for the model.

Service

App\Models\Service

- **user()**
- **plans()**
- **branch()**
- **factory(\$count, \$state)**

Get a new factory instance for the model.

Comandos de consola

PollPppoe

App\Console\Commands\PollPppoe

Muestrea el tráfico de las cuentas PPPoE de cada router.

- **__construct(\$poller)**
- **handle()**

TestWhatsAppTemplate

App\Console\Commands\TestWhatsAppTemplate

Envía una plantilla de prueba sin disparar eventos de negocio.

- **handle(\$gateway)**

Middleware

Authenticate

App\Http\Middleware\Authenticate

Sin métodos públicos propios.

EncryptCookies

App\Http\Middleware\EncryptCookies

Sin métodos públicos propios.

PreventRequestsDuringMaintenance

App\Http\Middleware\PreventRequestsDuringMaintenance

Sin métodos públicos propios.

RedirectIfAuthenticated

App\Http\Middleware\RedirectIfAuthenticated

- **handle(\$request, \$next, \$guards)**
Handle an incoming request.

TrimStrings

App\Http\Middleware\TrimStrings

Sin métodos públicos propios.

TrustHosts

App\Http\Middleware\TrustHosts

- **hosts()**
Get the host patterns that should be trusted.

TrustProxies

App\Http\Middleware\TrustProxies

Sin métodos públicos propios.

ValidateSignature

App\Http\Middleware\ValidateSignature

Sin métodos públicos propios.

VerifyCsrfToken

App\Http\Middleware\VerifyCsrfToken

Sin métodos públicos propios.

Proveedores de servicios

AppServiceProvider

App\Providers\AppServiceProvider

- **register()**
Register any application services.
- **boot()**
Bootstrap any application services.

AuthServiceProvider

App\Providers\AuthServiceProvider

- **boot()**
Register any authentication / authorization services.

BroadcastServiceProvider

App\Providers\BroadcastServiceProvider

- **boot()**

Bootstrap any application services.

EventServiceProvider

App\Providers\EventServiceProvider

- **boot()**
Register any events for your application.
- **shouldDiscoverEvents()**
Determine if events and listeners should be automatically discovered.

RouteServiceProvider

App\Providers\RouteServiceProvider

- **boot()**
Define your route model bindings, pattern filters, and other route configuration.

Utilidades

PdfBranding

App\Support\PdfBranding

Datos de marca para los PDFs del sistema.

- **make(\$view, \$data, \$landscape)**
Construye un PDF del sistema con el formato estándar: tamaño carta, orientación indicada y paginación "Página X de Y" en el pie.
- **branch(\$branch)**
Sucursal del documento: la indicada o, por defecto, la sucursal activa en sesión.
- **logoPath(\$branch)**
Ruta absoluta del logo de la sucursal, o null si no tiene logo cargado o el archivo no existe en disco.
- **locationLine(\$branch)**
Línea de ubicación de la sucursal (municipio, departamento, país) omitiendo los campos vacíos.
- **phoneLine(\$branch)**
Línea de teléfonos de la sucursal.

Filtros

UnreadOrdersBadgeFilter

App\Filters\UnreadOrdersBadgeFilter

Pone el número rojo de órdenes no vistas en el ítem "Mis Órdenes" del menú.

- **transform(\$item)**

Exportaciones

TransactionsReport

App\Exports\TransactionsReport

- **query()**
- **headings()**
- **map(\$transaction)**

- `download($fileName, $writerType, $headers)`
- `store($filePath, $disk, $writerType, $diskOptions)`
- `queue($filePath, $disk, $writerType, $diskOptions)`
- `raw($writerType)`
- `toResponse($request)`

Create an HTTP response that represents the object.

Escuchadores de eventos

PersistDarkModePreference

App\Listeners\PersistDarkModePreference

Guarda en el usuario el tema que acaba de elegir.

- `handle($event)`

5. Comandos de consola

Comandos propios del sistema. Los marcados como programados los ejecuta el planificador de Laravel automáticamente; el resto se lanzan a mano cuando la situación lo pide.

Comando	Para qué sirve	Cuándo
<code>php artisan onts:poll</code>	Muestrea por SNMP las métricas de las ONT y guarda el historial que alimenta las gráficas.	Programado cada 5 minutos
<code>php artisan onts:poll --resolve-traffic --prune=30</code>	Además resuelve los índices de tráfico y descarta muestras de más de 30 días.	Programado a las 03:30
<code>php artisan pppoe:poll</code>	Muestrea el consumo de las cuentas PPPoE en los routers MikroTik.	Programado cada 5 minutos
<code>php artisan pppoe:poll --prune=30</code>	Igual, depurando el historial antiguo.	Programado a las 03:45
<code>php artisan onts:import {olt}</code>	Importa a GestISP las ONT que ya estaban configuradas en una OLT. Admite <code>--preview</code> para ver qué traería sin escribir nada.	Manual, al incorporar una OLT existente
<code>php artisan onts:sync-power</code>	Actualiza la potencia óptica de todas las ONT.	Manual / diagnóstico
<code>php artisan olt:snmp-probe {olt}</code>	Verifica la conectividad SNMP y comprueba los OID configurados contra el equipo real. Con <code>--interfaces</code> muestra las descripciones de interfaz.	Manual, al configurar una OLT nueva
<code>php artisan olt:sync-interfaces</code>	Recalcula la correspondencia entre el índice SNMP y la posición física (slot/puerto) de las ONT.	Manual, tras cambios en la OLT
<code>php artisan invoices:notify-reminders</code>	Avisa al cliente cuando su factura está por vencer o ya venció.	Programado a las 08:00
<code>php artisan sessions:sweep</code>	Marca como expiradas las sesiones inactivas.	Programado cada hora
<code>php artisan permissions:sync</code>	Crea en la base los permisos declarados en los controladores y se los asigna al superadministrador. Admite <code>--dry-run</code> .	OBLIGATORIO en cada despliegue que agregue permisos
<code>php artisan audits:prune --days=N</code>	Depura la bitácora de auditoría conservando los últimos N días. Admite <code>--dry-run</code> . Mínimo 30 días.	Manual y deliberado: no está programado

5.1 El planificador

Para que las tareas programadas funcionen, el servidor debe tener esta única línea en el cron del usuario del servidor web:

```
* * * * * cd /var/www/gestisp && php artisan schedule:run >> /dev/null 2>&1
```

Sin esa línea no se muestrean las ONT ni las cuentas PPPoE, no se envían los avisos de factura y no se depuran las sesiones inactivas.

5.2 Comandos de Laravel de uso frecuente

Comando	Para qué sirve	Cuándo
<code>php artisan migrate</code>	Aplica las migraciones pendientes de base de datos. En producción se usa <code>--force</code> .	En cada despliegue
<code>php artisan test</code>	Ejecuta las 302 pruebas automatizadas sobre la base <code>gestisp_testing</code> .	Antes de cada entrega

Comando	Para qué sirve	Cuándo
php artisan queue:work	Procesa la cola (notificaciones, importación de ONT). En el servidor se gestiona con Supervisor.	Permanente si se usa cola
php artisan queue:restart	Obliga a los procesos de cola a recargar el código nuevo.	Tras cada despliegue
php artisan route:list	Lista las rutas registradas con su nombre y controlador.	Diagnóstico
php artisan tinker	Consola interactiva para inspeccionar datos y servicios.	Diagnóstico

6. Despliegue y operación

6.1 Procedimiento de despliegue

Pasos en orden. Los que dependen de que algo haya cambiado se indican entre paréntesis.

Paso	Comando
1. Traer los cambios	git pull
2. Dependencias (si cambiaron)	composer install --no-dev --optimize-autoloader
3. Recursos del navegador (si cambiaron)	npm install && npm run build
4. Base de datos	php artisan migrate --force
5. Permisos nuevos	php artisan permissions:sync
6. Limpiar y recachear	php artisan config:clear && php artisan view:clear && php artisan config:cache && php artisan route:cache && php artisan view:cache
7. Enlace de archivos públicos (solo la primera vez)	php artisan storage:link
8. Reiniciar la cola (si se usa)	php artisan queue:restart

6.2 Errores frecuentes al desplegar

- Una pantalla nueva responde 403: falta ejecutar php artisan permissions:sync.
- Un cambio de configuración no surte efecto: falta php artisan config:clear seguido de config:cache.
- La interfaz se ve rota o sin estilos: faltó npm run build, o php artisan view:clear.
- Las fotos e imágenes subidas no se ven: falta el enlace php artisan storage:link.
- Los trabajos en cola siguen con el código viejo: falta php artisan queue:restart.

6.3 Requisitos del servidor

Además de PHP 8.3, Apache y MySQL, el sistema necesita estas extensiones de PHP:

Extensión	Para qué
php-snmp	Monitoreo de OLT y ONT. Sin ella el módulo de red no funciona.
php-gd	PDF con imágenes y firmas, y códigos de barras.
php-zip y php-xml	Exportaciones a Excel.
php-mysql	Base de datos.
php-curl	Notificaciones y API de MikroTik.
php-bcmath	Cálculos de facturación.
php-mbstring	Manejo de texto con acentos.

7. Pruebas automatizadas

El proyecto tiene 302 pruebas repartidas en 38 archivos dentro de tests/Feature, agrupadas por módulo. Se ejecutan sobre una base de datos aparte (gestisp_testing) que se reconstruye en cada corrida.

```
php artisan test
php artisan test tests/Feature/Billing
```

Criterios que sigue el proyecto al escribir pruebas:

- Los nombres de las pruebas describen la regla de negocio en español, no el método que ejercitan.
- Se prueba también lo que NO debe ocurrir: accesos denegados, validaciones que rechazan, datos de otra sucursal que no deben verse.
- Nada depende de equipos de red reales: las comunicaciones con OLT y routers se sustituyen por dobles de prueba.
- Toda funcionalidad nueva llega con sus pruebas, y la suite completa debe quedar en verde antes de entregar.